Interactive Visual Gym: Evaluating Visual Interaction Models with Simulated Users in Interactive Worlds

Chenhao Zheng[1,2] **Zixian Ma**[1] **Kai Li**[2] **Kanchana Ranasinghe**[2]
Jieyu Zhang[1] **Artemis Panagopoulou**[2] **Weikai Huang**[1,2] **Jianxu Shangguan**[1]
Yuanyuan Tan **Silvio Savarese**[2] **Juan Carlos Niebles**[2] **Ranjay Krishna**[1]
Ran Xu[2] **Honglu Zhou**[2]

[1]University of Washington [2]Salesforce AI Research

Visual interaction models that support users during ongoing physical tasks require evaluation in closed loop, where assistant responses can change subsequent user behavior and task outcomes. Existing benchmarks largely rely on offline replay and therefore cannot capture these effects. We introduce **Interactive Visual Gym**, a framework for closed-loop evaluation and training of visual interaction models through joint simulation of executable task environments and human users. We contribute three simulators that recreate everyday tasks and human-aligned action spaces across cooking, household manipulation, and mobile-interface domains, together with a persona-conditioned user simulation harness that collaborates with the assistant while incorporating controlled execution errors and other out-of-plan behaviors. We instantiate the framework as **IVG-Bench**, a benchmark with 315 interaction scenarios. Evaluating state-of-the-art VLMs as visual interaction models reveals substantial limitations: the best-performing model, Claude Opus 5.5, completes only 53.5% of tasks within the time limit, exhibits significant weaknesses in factual grounding and actionable guidance, and is often too slow to prevent errors when response latency is considered. We verify our evaluation protocol with a human study which finds 78.5% agreement between our interaction-quality metric and participant preferences. We further show that the environments provide useful learning signals: in a proof-of-concept GRPO experiment, closed-loop training improves both task success and interaction quality for Qwen3.5-4B model in our cooking environment.

Project page: https://interactive-visual-gym.salesforceairesearch.com
Code: https://github.com/SalesforceAIResearch/Interactive-Visual-Gym

1 Introduction

Building systems that collaborate effectively with people is a longstanding goal of interactive machine learning (Amershi et al., 2014). Recent advances in LLM agents have enabled collaboration on digital tasks, including web research and software development, through dialogue, preference adaptation, and coordinated action (Shao et al., 2026; Sun et al., 2026). More recently, extending such collaboration to everyday physical tasks has drawn growing interest, motivating *visual interaction models* that observe a video stream of a user's ongoing activities and provide guidance (Cui et al., 2026; Thinking Machines Lab, 2026b; Yao et al., 2026). For example, an assistant monitoring a procedural physical task like cooking can help plan the next step, detect execution errors, and guide recovery if a mistake happens.

Developing and evaluating interactive agents requires closed-loop rollouts that capture how each interaction changes subsequent user behavior and environment state, enabling assessment of its contribution to task completion. For digital agents, existing websites and software provide readily available tasks and executable environments, which can be paired with simulated users to support such rollouts (Zhou et al., 2024; Yao et al., 2024; Shao et al., 2026). Building an equivalent testbed for visual assistance is substantially more difficult. First, physical tasks and their environments are not readily available in a digital form; they must be reconstructed with visual observations, underlying world state, and an action space that supports human task execution. Second, visual assistance places substantially greater demands on user simulation. In most digital settings, the model executes actions while the user primarily clarifies goals and preferences. In physical activities, the user is the acting agent, and the model influences execution through communication. The simulated user must plan, act, make plausible mistakes, while interpreting guidance and deciding how to respond. These simulation challenges have limited visual interaction model training and evaluation to replaying recorded interactions and matching reference responses (Liu et al., 2026; Kundu et al., 2026;

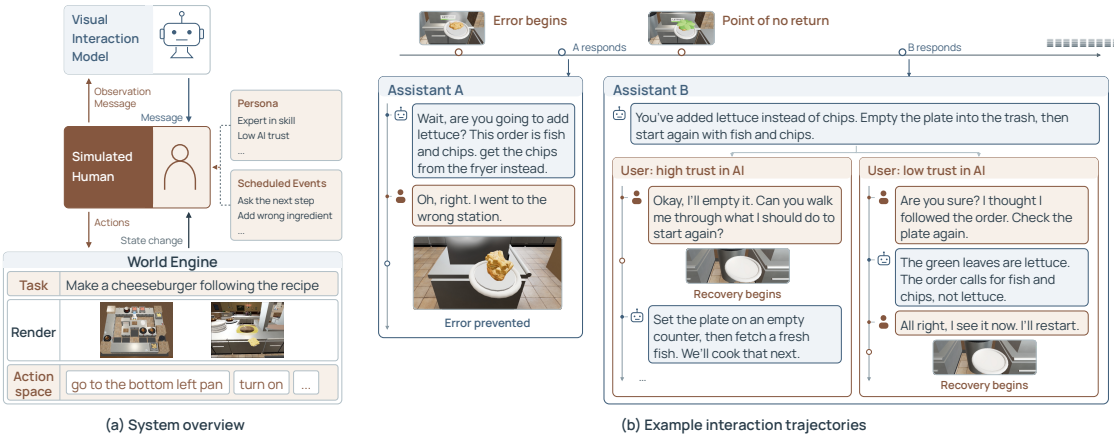


Figure 1: **Interactive Visual Gym.** (a) System overview: The visual interaction model under evaluation is tasked with guiding a simulated user to complete a physical task in a visual environment. (b) Examples showing how different task execution can be simulated based on different assistant response.

Ran et al., 2026), where the user's actions and responses remain unchanged regardless of the assistant's guidance.

We present **Interactive Visual Gym**, a framework for closed-loop evaluation of visual interaction models through joint simulation of task environments and human users. Our framework has three components:

First, we recreate everyday tasks across cooking, household manipulation, and mobile interface use in executable visual environments. These environments render visual observations from the user's perspective. They use scene graphs to represent world state and track task progress independently of variations in action execution. They expose a human-aligned semantic action space for simulated users to operate in, translating instructions such as "put the pan on the stove" into executable operations.

Second, we design a harness that turns a state-of-the-art VLM into simulated users that plan and act within these environments while interacting with the assistant and revising their actions in response to its guidance. The harness supports controlled insertion of common human out-of-plan events at specified task states, including repeating, forgetting, or incorrectly executing a step, changing plans, and asking questions. We further model diverse user personas through attributes such as skill level, trust in AI, and interruption preference. These attributes shape how users perform tasks and respond to guidance differently, enabling evaluation of the assistant's adaptation to different users.

Third, simulation provides execution traces that support comprehensive evaluation of an assistant, covering task completion, error detection, and interaction quality. This captures how assistance affects task execution and user experience, which offline replay cannot assess.

As an initial instantiation of our framework, we construct **IVG-Bench**, a benchmark with 150 cooking, 75 household, and 90 mobile-interface interaction scenarios to evaluate state-of-the-art VLMs as visual interaction models. In each episode, the assistant is tasked to support a simulated user with a sampled persona through task execution, and needs to deal with a set of scheduled out-of-plan events. The dynamic rollout allows us to measure task completion under assistance and assess individual responses against the underlying world state. The best-performing model, Claude Opus 5.5, completes only 53.5% of tasks within the time limit, averaged across the three environments, and also shows severe deficits in factual grounding and actionable guidance, revealing a substantial gap toward reliable interactive assistance.

Finally, we demonstrate that CookSim can support end-to-end reinforcement learning of multimodal assistants through a proof-of-concept GRPO experiment with Qwen3.5-4B (Qwen Team, 2026a) as the assistant and Gemini 3.7 Flash (Google, 2026b) as the simulated novice user. After 12 RL updates, the assistant's mean in-time task success rate improves by 8.9 percentage points on held-out development-validation scenarios with a Gemini 3.7 Flash user, and by 3.3 percentage points in a separate evaluation with a Gemini 3.8 Flash user (Tab. 5). Despite limited tasks and training updates, these results show that CookSim can supply a learning signal and induce policy improvements in interactive task performance.

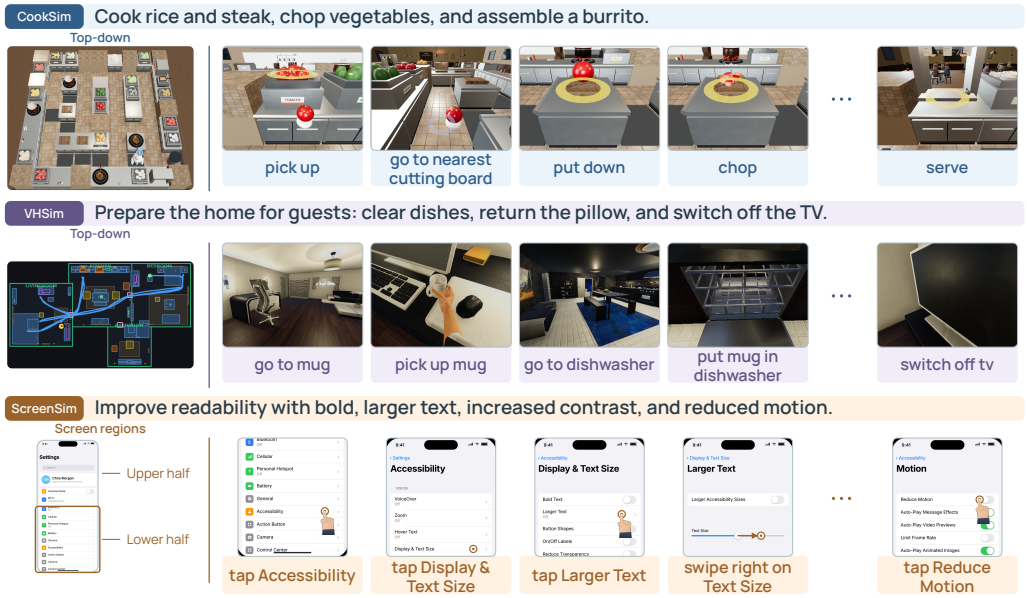


Figure 2: **Example tasks in our environments.** We show the simulated tasks across three domains, with representative action commands and corresponding visual observations during task execution. Our semantic action spaces express actions at the level of everyday human communication.

2 Related Work

Streaming visual assistance. Prior work develops multimodal representations through camera-metadata supervision (Zheng et al., 2023), compositional learning (Zheng et al., 2024), and trajectory-based video tokenization (Zheng et al., 2025; 2026). Motion forecasting extends this focus from understanding observed activity to predicting goal-conditioned object trajectories (Zhang et al., 2026). Recent interaction models continuously process multimodal streams and support low-latency, proactive, or full-duplex responses (Cui et al., 2026; Yao et al., 2026; Orantqing et al., 2026). Live guidance has also been studied through user-in-the-loop cooking assistance (Verghese et al., 2024) and workspace-conditioned visual instructions (Wu et al., 2026). Complementary datasets and benchmarks study procedural guidance, mistake recovery, correct silence, and response timing in egocentric or general video streams (Zhang et al., 2025a; Liu et al., 2026; Kundu et al., 2026; Bhattacharyya et al., 2026; Chatterjee et al., 2026; Vasu et al., 2026). In these benchmarks a candidate model is typically scored against a prerecorded trajectory. Our framework instead evaluates response-contingent rollouts.

Interactive agent environments. Embodied benchmarks place agents in simulated worlds for navigation, manipulation, and collaboration (Shridhar et al., 2020; Puig et al., 2018; Li et al., 2022; Padmakumar et al., 2022; Puig et al., 2024; Chang et al., 2025), and recent multimodal gyms unify diverse visual tasks for evaluation and reinforcement learning (Wang et al., 2026; Meng et al., 2026) or study which environment mixtures best train visual agents (Zhu et al., 2026); in all of these, the evaluated model selects the actions itself. Digital interaction benchmarks instead pair tool-using agents with simulated users and verify outcomes from application state (Yao et al., 2024; Barres et al., 2025; Shao et al., 2026), which τ-Voice extends to voice interaction (Ray et al., 2026). BASIS trains and evaluates visual interaction models through simulated user interactions in Alexa Arena (Zhang et al., 2025b). User simulation has long enabled scalable dialogue evaluation (Schatzmann et al., 2005), and language models now support persona-conditioned users (Aher et al., 2023), but simulated users can diverge systematically from people, requesting validation (Seshadri et al., 2026).

3 Method

Our framework combines executable physical task environments with simulated human users to evaluate visual interaction models. We first describe how these environments reconstruct everyday tasks, expose human-communicable actions, and track task progress. We then introduce the human-agent harness and its interaction with the assistant, followed by the evaluation metrics.

3.1 EXECUTABLE PHYSICAL TASK ENVIRONMENTS

Simulating everyday tasks. We reconstruct procedural physical tasks in three domains (Fig. 2):

Cooking. Inspired by Overcooked (Ghost Town Games, 2016), we build CookSim from scratch, with recipes specifying ingredient quantities, preparation requirements, and plating order where applicable. Users navigate kitchens with distributed ingredients and tools while accomplishing the cooking task. For example, preparing a cheeseburger requires cooking a patty and plating the bun, patty, and cheese in that order. The simulator is implemented in Three.js/WebGL and supports top-down and egocentric 3D rendering.

Household activities. We adapt VirtualHome (Puig et al., 2018) for everyday apartment tasks combining navigation and object manipulation. For example, cleaning up after a meal requires loading plates and a mug into the dishwasher, returning an apple to the fridge, and closing both appliances. The simulator uses VirtualHome's Unity backend for egocentric 3D rendering, while providing a top-down trajectory map.

Mobile interface use. ScreenSim replicates the iPhone 17 Settings interface, with multi-step configuration tasks derived from Apple's user guides (Apple, 2026). For example, restricting an app's access requires disabling its location permission, cross-app tracking, and background refresh while leaving other apps unchanged. The simulator is implemented in HTML/CSS rendered in Chromium and captures mobile-screen observations through Playwright.

Human-communicable action space. Existing physical-task simulators expose simulator-specific movement and interaction primitives, such as movement increments or object identifiers (Li et al., 2022; Puig et al., 2018; Fan et al., 2022). People typically communicate at a more abstract semantic level, referring to objects and intended operations. We therefore build a semantic action space for each environment (Fig. 2), with commands such as "go to the nearest pan" in CookSim, "place the book on the table" in VHSim, and "select Background App Refresh" in ScreenSim. During execution, an LLM-based grounding module resolves contextual references, such as "the pan on your left," into object identifiers recognized by the simulator. The simulator then expands each high-level action into a sequence of low-level controls that update the world state and generate visual observations.

Progress tracking via scene-graph state. Each simulator represents world state as a scene graph with nodes encoding objects and attributes, such as a dishwasher's open or closed state, and edges encoding containment and support relations. We decompose each task into sub-goals defined by conditions over partial graph state, such as the designated dishes being inside the dishwasher. Tracking these conditions measures task progress independently of different valid execution procedures.

3.2 USER–ASSISTANT INTERACTION

The visual interaction model under evaluation is tasked with guiding a simulated user through tasks in the environments described above. We implement the user as a VLM agent with access to structured world-state information, execution feedback, and conversation history. The VLM is prompted to role-play a human user to test the assistant's interaction capabilities, planning and acting independently while responding to guidance according to its persona. A user-simulation harness conditions its behavior through persona instructions and temporarily takes control of its actions to enact scheduled out-of-plan events. Below, we describe the action and communication interfaces, account for assistant response delays, and detail these mechanisms for controlling user behavior.

Action interface: user–environment. The simulated user maintains an action queue that the engine executes at each simulation tick and can revise in response to environment changes or assistant suggestions. The engine updates the world state and renders the resulting observation:

$$s_{t+1} = T(s_t, a_t), \qquad o_{t+1} = R(s_{t+1}), \tag{1}$$

where a_t is the action executed from the user's queue, T is the state-transition, and R is the renderer.

Communication interface: user–assistant. The assistant monitors sampled frames from the user's observation stream alongside the conversation history. At an assistant invocation at tick t, it generates a response

$$m_t = A(o_{\leq t}, c_{\leq t}), \tag{2}$$

where $o_{\leq t}$ denotes the sampled visual history and $c_{\leq t}$ is the conversation history. The response m_t contains an utterance or silence in text format and an optional event-type flag (Tab. 1(A)), which is enabled when

A Scheduled out-of-plan events (CookSim)

Category	Event types
Execution errors	Repeat, forget, or skip a step; wrong plating order; wrong ingredient; add unwanted ingredients; plate raw food; unattended cooking.
Questions	Action order; counts; elapsed time; time duration; event recall
Proactive requests	Notify when a specified condition holds.
Plan changes	Modify the recipe; perform a side task.

B User personas

Axis	Levels / behavior
Skill	Low / Medium / High *Planning and anticipation.*
Trust in AI	Low / Medium / High *Evidence needed to accept advice.*
Execution speed	Cautious / Balanced / Quick *Careful versus rapid execution.*
Interruption	Step-by-step / Moderate / Necessary only *Receptiveness to frequent guidance.*

Table 1: **Controllable user simulation.** (A) Scheduled out-of-plan events, illustrated with CookSim, test assistant responses to different types of out-of-plan events. Events are triggered by conditions on task progress. (B) Four persona axes, each with three levels, simulate how different simulated users perform tasks and respond to guidance differently .

the assistant detects a user execution error. On each user turn, the simulated user likewise produces an utterance or remains silent. Its conversational response types include asking for an explanation, questioning the correctness of guidance, requesting more guidance or fewer interruptions, accepting or rejecting a suggestion, and answering a question.

Accounting for assistant delay. Assistant processing delay can substantially affect interaction quality: by the time a response arrives, the user and environment may have progressed beyond the observed state, rendering the guidance outdated or invalid. Following the separation of simulation time and wall-clock time in τ-Voice (Ray et al., 2026), we assign each simulation tick a duration Δ based on approximate real-world action speeds. For example, VirtualHome is configured to move the user one meter per simulation tick. Assuming this distance takes 1.2 seconds to walk in the real world, we assign each tick a duration of $\Delta = 1.2$ seconds. We convert the measured assistant inference latency ℓ_t into a delay of $d_t = \lceil \ell_t / \Delta \rceil$ simulation ticks. The response m_t, generated from observations through tick t, is delivered at tick $t + d_t$. During this interval, the engine continues executing the user's queued actions and updating the world state.

Scheduled out-of-plan events. We introduce scheduled out-of-plan events in all three simulators that require the assistant to respond to execution errors, questions and intended plan change. For example, the event types in CookSim are shown in Tab. 1(A): we simulate common cooking errors of incorrect preparation, omitted or improperly sequenced steps, and ill-timed actions (Peddi et al., 2024); we generate temporal questions adapted from streaming-video benchmarks (Niu et al., 2025; Huang et al., 2025; Lin et al., 2026) and proactive requests for timely assistance (Zhang et al., 2025a); we also introduce intended plan changes, where users may announce revised goals or a side task during execution. Each event is triggered by a specified condition on task progress.

When an event is triggered, the harness temporarily takes control of the user's action queue to enforce its execution. It privately informs the user agent of the ongoing event and explains that the event is intended to test the assistant, instructing the user agent to adapt its conversation accordingly. For mistake events, the user agent judges whether the assistant has correctly identified the mistake and decides whether to accept its guidance. Once the guidance is accepted, the harness returns control of the action queue to the user agent.

Persona-conditioned users. Real-world users differ in how they perform tasks and respond to assistance. Drawing on HCI and related behavioral research, we operationalize this diversity through four representative axes: skill level (Shneiderman, 2000), trust in AI (Lee & See, 2004), execution speed (Wickelgren, 1977), and interruption preference (McFarlane, 2002) (Tab. 1(B)). The harness instantiates each persona through axis-specific instructions in the user model's context. For example, high-skill users are prompted to plan ahead independently, while low-skill users are prompted to ask assistant frequently to determine the next steps. Users who prefer minimal interruption are prompted to find unnecessary commentary disruptive, whereas those who benefit from step-by-step support prefer more frequent guidance. The execution speed and interruption preference are shared to assistant as part of announced user preference, while the skill level and trust in AI are treated as internal property of a user for assistants to infer and adapt.

3.3 Evaluation Metrics

Our simulation rollouts allow us to measure task completion under assistance and assess individual responses against the underlying world state. We evaluate the assistants with the following metrics:

In-time success. *In-time success* measures the proportion of episodes whose engine-verified goal is satisfied within a completion budget of $2(T_i^{\text{ref}} + \alpha_i)$, where T_i^{ref} is the duration of an engine-generated, error-free reference rollout and α_i is the prescribed event allowance. Environment-specific budgets are detailed in Appendix D.

Error detection. *Detection recall* is the proportion of triggered errors correctly identified within their eligible detection windows, credited when the assistant raises a correct event flag verified by the engine or when the simulated user judges its diagnosis correct. For the latency ablation, we also report *error prevention*, the proportion of triggered errors prevented before the point of no return. Prevention requires both timely identification and guidance that persuades the user to change course.

Interaction quality. We evaluate each assistant response against the underlying world state, rather than against a reference response as in prior works. We define five dimensions of evaluation informed by dialogue evaluation (Mehri & Eskenazi, 2020) and HCI guidance on context-sensitive, user-controllable assistance (Amershi et al., 2019): factual grounding, situational relevance, actionable guidance, user intent uptake, and guidance conciseness. We operationalize these dimensions as binary rubric items evaluated by two LLM judges (Liu et al., 2023; Lee et al., 2025). For each conversational round, each judge receives engine state, user preferences, conversation history, and subsequent user actions, returning evidence-supported pass, fail, or not-applicable judgments of each rubrics. A category passes only when all applicable items pass; each judge's overall score averages applicable categories, gated by its factual-grounding judgment. We average the two judges' round scores before aggregating across rounds and runs. Judge prompts and score aggregation are detailed in Appendix I.

Question answering and proactive requests. The engine pairs each generated question with a grading procedure that derives reference facts from world state and execution history. A semantic judge assigns correctness against these facts. For proactive requests, the engine derives an acceptable notification window from the relevant state transitions.

4 Experiments

We use our engine to instantiate **IVG-Bench** benchmark for evaluating state-of-the-art VLMs as visual interaction models (Sec. 4.1), and validate our evaluation paradigm through a human study (Sec. 4.2). We further present a preliminary reinforcement learning experiment showing that our engine can support training visual interaction models and improve their interaction performance (Sec. 4.3).

4.1 IVG-Bench: Benchmarking Visual Interaction Models in Interactive Visual Gym

IVG-Bench comprises 315 interaction scenarios: 150 in CookSim, 75 in VHSim, and 90 in ScreenSim. Each case specifies a task, a user persona, and scheduled out-of-plan events triggered by task-progress conditions. Below we detail the setting for evaluated visual interaction models and show evaluation results.

4.1.1 Visual Interaction Model Setting

Models. We evaluate four classes of assistants: (1) frontier API models: GPT-6 Astra (OpenAI, 2026a), GPT-6.1 Sol (OpenAI, 2026c), Gemini 3.8 Flash (Google, 2026c), Gemini 4 Argon (Kavukcuoglu, 2026), and Claude Opus 5.5 (Anthropic, 2026); (2) real-time streaming API models: Gemini 3.8 Live (Google, 2026d) and GPT-Realtime 2.1 (OpenAI, 2026b); (3) open-weight non-streaming VLMs: Qwen3.8-27B (Qwen Team, 2026b), Molmo2-8B (Clark et al., 2026), InternVideo3-8B (Yan et al., 2026b), Kimi-VL-A3B-Thinking (Kimi Team et al., 2025), Muse Glimmer (Meta, 2026), and Inkling (Thinking Machines Lab, 2026a); and (4) open-weight streaming VLMs: MiniCPM-o 4.5 (Cui et al., 2026), Qwen3-Omni-30B (Xu et al., 2025a), Proact-VL (Yan et al., 2026a), and StreamingVLM (Xu et al., 2025b). We use Gemini 3.7 Flash as the simulated user in CookSim and ScreenSim, and Gemini 3.1 Pro (Google, 2026a) in VHSim. We examine sensitivity to the user model in Appendix A.

Inference. Assistants observe the user's ongoing activity through visual observations and conversation, without access to the scheduled events or the user's private action plan. In CookSim and VHSim, we query

Assistant	CookSim								VHSim								ScreenSim							
	In-time success	Det. recall	Interaction quality						In-time success	Det. recall	Interaction quality						In-time success	Det. recall	Interaction quality					
			Fac. grnd.	Sit. rel.	Act. guid.	User intent	Guid. conc.	Overall			Fac. grnd.	Sit. rel.	Act. guid.	User intent	Guid. conc.	Overall			Fac. grnd.	Sit. rel.	Act. guid.	User intent	Guid. conc.	Overall
Frontier API models																								
Gemini 4 Argon	**60.9**	80.2	70.2	90.1	71.4	82.2	76.9	63.8	**15.6**	**81.2**	60.3	90.7	60.8	83.2	84.8	55.6	83.7	**100.0**	91.0	95.5	86.5	87.1	95.1	87.1
Claude Opus 5.5	57.1	**86.0**	69.3	92.4	75.3	83.8	90.7	66.0	14.2	62.6	71.8	**93.8**	**65.1**	88.4	91.4	67.0	89.3	**100.0**	87.1	96.5	90.0	85.9	95.3	84.0
Gemini 3.8 Flash	46.4	77.9	**83.5**	83.5	65.5	66.2	90.9	73.4	14.7	72.2	79.2	88.4	63.4	75.3	**92.7**	**71.2**	78.5	97.8	92.4	95.7	89.4	80.2	**99.0**	**89.4**
GPT-6.1 Sol	42.9	79.9	80.4	92.4	78.1	91.1	**91.7**	75.8	**15.6**	60.3	77.8	92.9	57.2	89.1	92.2	70.5	87.4	**100.0**	89.0	96.6	**91.7**	**93.4**	97.3	86.5
GPT-6 Astra	40.2	80.4	80.5	**92.9**	**78.9**	**93.6**	90.5	**76.0**	10.7	62.1	77.7	92.8	59.5	**90.7**	91.4	71.1	**90.4**	**100.0**	89.0	**96.8**	91.0	91.5	96.5	86.3
Real-time streaming API models																								
GPT-Realtime 2.1	33.3	68.0	65.9	69.4	46.2	44.8	37.3	45.1	11.6	60.8	79.2	86.9	64.3	67.4	56.5	62.3	66.7	98.6	90.5	96.2	85.3	88.4	86.3	84.5
Gemini 3.8 Live	26.2	58.5	76.8	71.4	44.6	55.1	81.8	60.6	11.6	24.6	**83.5**	81.5	49.8	58.5	83.8	67.4	71.9	96.1	**92.5**	95.1	86.8	72.2	98.2	89.1
Open-weight non-streaming VLMs																								
Muse Glimmer	40.7	60.5	28.9	68.6	33.7	36.5	27.6	21.6	12.4	14.3	53.8	83.0	50.0	61.7	50.4	43.6	44.1	96.1	61.0	83.2	61.7	55.0	66.4	53.8
Inkling	30.4	79.5	52.7	72.5	39.2	44.6	68.2	41.3	8.0	79.0	62.8	86.2	48.4	63.7	83.5	53.9	47.8	98.4	83.9	89.6	73.7	51.5	92.0	77.0
Qwen3.8-27B	29.8	60.6	51.9	83.8	53.7	70.1	68.1	44.6	12.0	31.8	54.3	88.0	56.3	79.8	74.9	48.7	67.8	96.2	88.7	95.7	89.0	72.7	97.0	86.2
InternVideo3-8B	7.3	25.6	46.5	36.3	16.7	17.1	48.3	27.7	4.0	0.0	48.7	54.7	31.2	26.0	45.1	32.6	0.4	41.1	51.0	41.9	22.5	14.7	62.6	33.9
Kimi-VL-Thinking	7.3	56.6	7.3	27.4	5.5	14.5	11.2	4.3	5.8	74.0	26.7	50.6	17.4	25.2	31.1	17.2	9.6	80.0	50.2	48.7	25.4	20.9	50.4	33.0
Molmo2-8B	5.6	5.1	41.9	18.5	10.0	14.6	12.3	16.9	5.8	1.1	54.4	41.3	23.0	13.0	32.1	30.3	0.7	21.5	74.0	27.1	17.6	12.3	59.7	39.2
Open-weight streaming VLMs																								
Qwen3-Omni-30B	23.6	39.0	40.7	52.9	28.8	24.7	47.4	28.3	10.7	2.7	56.9	73.4	41.8	39.8	67.3	44.4	15.2	58.3	71.1	68.8	51.7	28.5	80.0	58.2
MiniCPM-o 4.5	11.1	39.6	29.0	31.9	9.8	22.1	34.5	15.8	5.8	9.8	45.3	53.8	18.7	34.2	35.3	27.7	2.6	45.2	56.8	52.2	27.6	43.2	59.3	38.5
StreamingVLM	5.3	0.7	73.9	6.6	16.8	30.7	14.9	27.2	5.3	0.0	68.4	14.0	7.7	22.8	15.8	29.3	0.0	0.9	89.8	9.6	12.7	16.7	22.7	37.2
Proact-VL	2.4	0.8	43.5	10.5	5.1	13.8	23.7	17.8	2.2	0.0	58.5	23.1	7.0	16.3	30.5	27.1	0.0	1.9	72.8	6.5	3.0	3.4	31.6	27.9

Table 2: **Quantitative results across three environments.** Results are averaged across three independent runs. Det. recall: error detection recall; Fac. grnd.: factual grounding; Sit. rel.: situational relevance; Act. guid.: actionable guidance; User intent: user intent uptake; Guid. conc.: guidance conciseness. Scores are percentages; bold and underline indicate the best and second-best scores in each column.

assistants every two simulation ticks, including real-time API models in a polled setting. Streaming models retain context within their sessions. We adapt non-streaming models with a sliding window of the eight most recent observations and a recurrent, text-based Plan–Act memory. This follows the use of persistent procedural plans for visual assistance in Kundu et al. (2026). Each locally hosted open-weight model is served on an H100 GPU in CookSim and VHSim. ScreenSim settings are detailed in Appendix H.

Response timing. We use fixed response timing for the main comparison: two simulation ticks for CookSim and VHSim's streaming interactions, and zero ticks for ScreenSim's action–response interactions. We separately evaluate the effect of measured assistant latency in CookSim and VHSim. Each setting uses three independent rollouts per benchmark scenario.

4.1.2 Quantitative Results

We report task success, error detection, and interaction quality across the three environments in Tab. 2 and Fig. 3. Tabs. 3 and 4 report question answering and the effect of assistant latency, respectively. We discuss the main findings below.

Even the strongest VLMs fall short of reliable visual assistance. Claude Opus 5.5 achieves the highest task success, but completes only 53.5% of tasks within the time limit, averaged across the three environments. Moreover, Claude Opus 5.5 trails Astra, Sol, and Gemini Flash in interaction quality, with notable limitations in factual grounding and actionable guidance. *Open-weight assistants generally fall short of proprietary models on both task success and interaction quality.*

The strongest streaming models remain competitive in interaction quality but lag in task success. Gemini Live achieves high average interaction quality, yet completes fewer tasks than Astra and Gemini Flash. This pattern may reflect a trade-off between low-latency inference and high planning and reasoning ability.

Assistant	Question answering				Proactive request	Overall
	Temp. order	Act. count	Time dur.	Event recall		
Frontier API models						
Gemini 4 Argon	66.8	52.8	82.3	85.8	85.3	79.0
GPT-6 Astra	51.1	34.8	89.5	88.9	86.8	78.6
GPT-6.1 Sol	58.9	22.9	81.7	76.5	88.2	74.3
Claude Opus 5.5	62.5	28.3	75.8	69.1	62.7	63.3
Gemini 3.8 Flash	67.3	30.0	59.1	55.8	69.7	59.4
Real-time streaming API models						
Gemini 3.8 Live	46.9	23.5	51.5	73.5	43.4	47.6
GPT-Realtime 2.1	39.1	13.3	58.2	62.5	43.7	46.5
Open-weight non-streaming VLMs						
Qwen3.8-27B	47.1	18.2	49.8	64.9	25.8	38.7
Muse Glimmer	9.9	1.8	38.8	31.2	28.5	27.5
Inkling	9.2	9.2	36.4	8.8	36.4	27.4
Kimi-VL-Thinking	3.0	0.0	29.7	0.0	26.7	19.2
Molmo2-8B	9.7	1.9	15.3	38.1	14.3	15.8
InternVideo3-8B	0.0	0.0	17.4	8.3	21.8	14.3
Open-weight streaming VLMs						
Qwen3-Omni-30B	7.5	3.2	30.1	10.2	30.9	22.9
MiniCPM-o 4.5	5.6	0.0	9.3	19.2	9.6	9.6
StreamingVLM	2.2	1.8	2.3	2.8	5.3	3.4
Proact-VL	0.0	1.9	0.8	0.0	2.3	1.3

Table 3: **Question answering and proactive requests in CookSim.** Results are averaged across three runs with measured latency. Temp. order: temporal order; Act. count: action count; Time dur.: duration.

Higher task success does not necessarily imply better interaction quality. Although the two measures are positively associated (Fig. 3), task outcomes alone can hide weak assistance. For example, Muse Glimmer completes more tasks than Astra in CookSim while receiving much lower interaction-quality scores. Completing the task is therefore not sufficient evidence of high quality guidance.

Temporal question answering is surprisingly difficult during task-directed interaction. We present the result of temporal question answering from the measured-delay CookSim runs in Tab. 3. Unlike questions about fixed videos, the questions are asked by human agent at specific moments during multi-turn interactions in which the assistant must also help the user complete the task, closely reflecting real world scenario. We find that open-weight models often lose track of the question or request during multi-turn conversations (Tab. 3). Proprietary models generally perform better, with Gemini 4 Argon and Astra leading the pooled evaluation but still exhibit weakness in temporal action counting.

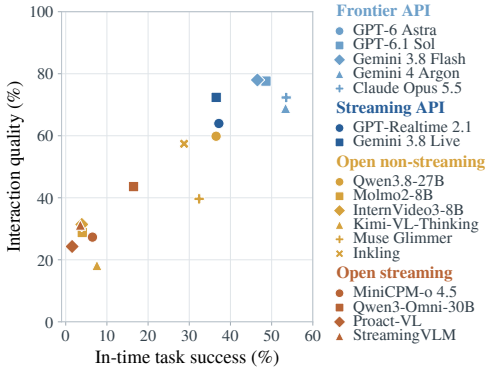


Figure 3: **Task success versus interaction quality.** Scores are averaged over all three environments.

4.1.3 Effect of Assistant Latency

We assess the effect of assistant response latency in CookSim and VHSim by comparing rollouts with measured latency against the fixed two-tick setting (Tab. 4 and Fig. 4). We use the same tasks, personas, and scheduled events in both settings, each averaged across three independent runs.

Response latency substantially reduces in-time task success for frontier API models. Introducing measured latency lowers success for all five frontier API models in both environments. The accompanying

Assistant	CookSim In-time success	CookSim Det. recall	CookSim Interaction quality: Fac. grnd.	Sit. rel.	Act. guid.	User intent	Guid. conc.	Overall	VHSim In-time success	VHSim Det. recall	VHSim Interaction quality: Fac. grnd.	Sit. rel.	Act. guid.	User intent	Guid. conc.	Overall
Frontier API models																
Gemini 4 Argon	33.1 (-27.8)	66.9 (-13.3)	65.3 (-4.9)	84.9 (-5.2)	63.0 (-8.4)	86.0 (+3.7)	71.1 (-5.8)	57.9 (-6.0)	**12.9** (-2.7)	**71.4** (-9.7)	49.5 (-10.7)	79.1 (-11.6)	43.7 (-17.1)	77.0 (-6.2)	76.0 (-8.8)	43.1 (-12.4)
Claude Opus 5.5	31.8 (-25.3)	64.2 (-21.7)	52.3 (-17.0)	81.2 (-11.2)	52.5 (-22.8)	85.7 (+1.9)	79.2 (-11.5)	46.9 (-19.1)	5.3 (-8.9)	54.9 (-7.7)	60.0 (-11.8)	85.0 (-8.8)	47.5 (-17.6)	83.8 (-4.6)	85.3 (-6.1)	53.8 (-13.2)
Gemini 3.8 Flash	29.8 (-16.7)	61.3 (-16.6)	77.6 (-5.9)	77.8 (-5.7)	58.5 (-7.0)	66.9 (+0.7)	88.4 (-2.5)	66.9 (-6.5)	10.7 (-4.0)	57.8 (-14.4)	70.0 (-9.2)	82.2 (-6.2)	**53.2** (-10.2)	72.2 (-3.2)	**90.2** (-2.5)	61.4 (-9.7)
GPT-6.1 Sol	30.0 (-12.9)	**67.5** (-12.5)	74.8 (-5.5)	**87.6** (-4.8)	69.2 (-8.9)	90.6 (-0.6)	86.3 (-5.4)	**68.4** (-7.5)	8.0 (-7.6)	59.0 (-1.2)	71.7 (-6.1)	**86.2** (-6.7)	44.3 (-13.0)	84.0 (-5.1)	88.3 (-3.9)	62.4 (-8.1)
GPT-6 Astra	28.4 (-11.8)	63.6 (-16.7)	74.0 (-6.5)	**87.6** (-5.3)	**69.6** (-9.3)	**92.4** (-1.2)	**88.5** (-2.0)	**68.4** (-7.5)	8.0 (-2.7)	47.9 (-14.2)	71.0 (-6.6)	85.7 (-7.1)	47.2 (-12.3)	**85.4** (-5.3)	89.3 (-2.2)	63.0 (-8.1)
Real-time streaming API models																
GPT-Realtime 2.1	22.0 (-11.3)	60.6 (-7.4)	66.1 (+0.1)	70.7 (+1.3)	46.9 (+0.7)	54.2 (+9.4)	44.0 (+6.8)	47.1 (+2.0)	10.2 (-1.3)	35.5 (-25.3)	72.7 (-6.6)	80.4 (-6.5)	49.9 (-14.4)	71.8 (+4.4)	56.9 (+0.4)	54.9 (-7.3)
Gemini 3.8 Live	22.2 (-4.0)	53.0 (-5.4)	**77.7** (+0.9)	74.0 (+2.6)	46.3 (+1.7)	61.0 (+5.9)	84.9 (+3.2)	62.4 (+1.8)	10.2 (-1.3)	19.8 (-4.8)	**81.7** (-1.8)	78.8 (-2.7)	46.4 (-3.3)	58.7 (+0.2)	85.5 (+1.7)	**65.6** (-1.9)
Open-weight non-streaming VLMs																
Muse Glimmer	**42.7** (+2.0)	60.7 (+0.2)	28.5 (-0.4)	67.5 (-1.1)	32.6 (-1.1)	36.0 (-0.5)	27.5 (-0.1)	21.2 (-0.3)	9.3 (-3.1)	13.9 (-0.4)	52.8 (-1.0)	82.1 (-1.0)	48.2 (-1.8)	65.4 (+3.7)	50.8 (+0.5)	43.0 (-0.6)
Inkling	8.2 (-22.2)	38.1 (-41.4)	50.4 (-2.3)	60.9 (-11.6)	29.5 (-9.6)	52.2 (+7.7)	67.6 (-0.6)	37.5 (-3.8)	9.3 (+1.3)	49.8 (-29.2)	55.4 (-7.4)	75.9 (-10.2)	33.4 (-14.9)	60.2 (-3.6)	77.8 (-5.7)	44.0 (-9.9)
Qwen3.8-27B	27.1 (-2.7)	57.1 (-3.6)	50.4 (-1.4)	81.4 (-2.4)	50.9 (-2.8)	70.2 (+0.1)	66.4 (-1.7)	43.0 (-1.6)	8.9 (-3.1)	30.4 (-1.4)	49.2 (-5.1)	83.2 (-4.9)	48.3 (-8.0)	77.9 (-1.9)	72.3 (-2.6)	43.5 (-5.2)
InternVideo3-8B	4.9 (-2.4)	14.7 (-10.8)	44.9 (-1.6)	33.9 (-2.4)	16.3 (-0.5)	18.1 (+1.0)	52.7 (+4.4)	27.7 (0.0)	7.1 (+3.1)	0.0 (0.0)	44.5 (-4.2)	52.5 (-2.1)	28.9 (-2.3)	29.1 (+3.1)	47.9 (+2.8)	30.6 (-2.0)
Kimi-VL-Thinking	6.2 (-1.1)	46.1 (-10.4)	8.2 (+0.9)	26.7 (-0.7)	5.4 (-0.2)	14.0 (-0.5)	12.2 (+0.9)	4.8 (+0.4)	11.1 (+5.3)	68.3 (-5.7)	27.9 (+1.2)	52.4 (+1.8)	17.9 (+0.5)	29.0 (+3.8)	32.0 (+0.8)	18.2 (+0.9)
Molmo2-8B	6.0 (+0.4)	4.5 (-0.6)	40.1 (-1.8)	18.7 (+0.2)	9.9 (-0.1)	15.6 (+1.0)	14.2 (+1.9)	16.8 (-0.1)	4.9 (-0.9)	0.0 (-1.1)	52.7 (-1.8)	39.9 (-1.4)	21.4 (-1.5)	12.9 (-0.2)	32.5 (+0.4)	29.5 (-0.8)
Open-weight streaming VLMs																
Qwen3-Omni-30B	22.4 (-1.1)	42.5 (+3.5)	39.4 (-1.3)	51.3 (-1.7)	28.0 (-0.8)	23.7 (-1.1)	47.0 (-0.4)	27.3 (-0.9)	6.7 (-4.0)	4.9 (+2.2)	57.5 (+0.6)	75.1 (+1.6)	42.2 (+0.4)	41.1 (+1.3)	69.7 (+2.4)	45.1 (+0.7)
MiniCPM-o 4.5	5.6 (-5.6)	36.3 (-3.3)	28.0 (-1.1)	29.6 (-2.3)	9.2 (-0.6)	21.1 (-1.0)	36.6 (+2.1)	15.1 (-0.7)	7.1 (+1.3)	11.2 (+1.4)	47.5 (+2.1)	55.1 (+1.3)	20.7 (+2.0)	37.0 (+2.8)	34.3 (-0.9)	29.1 (+1.4)
StreamingVLM	3.3 (-2.0)	1.0 (+0.4)	73.3 (-0.6)	7.1 (+0.4)	16.7 (-0.2)	29.1 (-1.6)	15.5 (+0.6)	27.2 (0.0)	3.1 (-2.2)	0.0 (0.0)	68.1 (-0.3)	14.4 (+0.3)	9.8 (+2.1)	24.4 (+1.6)	15.1 (-0.7)	29.0 (-0.3)
Proact-VL	3.3 (+0.9)	1.0 (+0.2)	45.0 (+1.5)	10.2 (-0.3)	5.5 (+0.4)	10.9 (-3.0)	23.6 (-0.2)	18.1 (+0.3)	3.6 (+1.3)	0.0 (0.0)	58.5 (+0.1)	22.7 (-0.4)	7.0 (0.0)	15.9 (-0.4)	29.3 (-1.2)	27.0 (-0.1)

Table 4: **Results with measured assistant latency.** Parentheses show changes from the fixed two-tick setting in percentage points. Results are averaged across three independent runs.

decline in error prevention helps explain this loss (Tab. 20): even when an assistant recognizes an impending mistake, its warning may arrive after the user has committed the action, incurring either task failure or additional steps to recover from errors.

Accounting for response latency changes model rankings. Gemini 4 Argon achieves the highest task success in CookSim under fixed response timing, while Muse Glimmer leads when measured latency is considered. In CookSim, Gemini 4 Argon's success falls from 60.9% to 33.1%, while Muse Glimmer changes little. Measured latency also lowers interaction quality for the frontier API models in both environments, particularly in situational relevance and actionable guidance.

4.2 Validating Interactive Visual Gym via Human Study

To examine both evaluation validity and user-simulation fidelity, we conduct a human study in CookSim simulator. 45 participants interact with 3 anonymously labeled assistants and rank the interaction experience after completing for 3 rounds. Full study protocol provided in Appendix F.

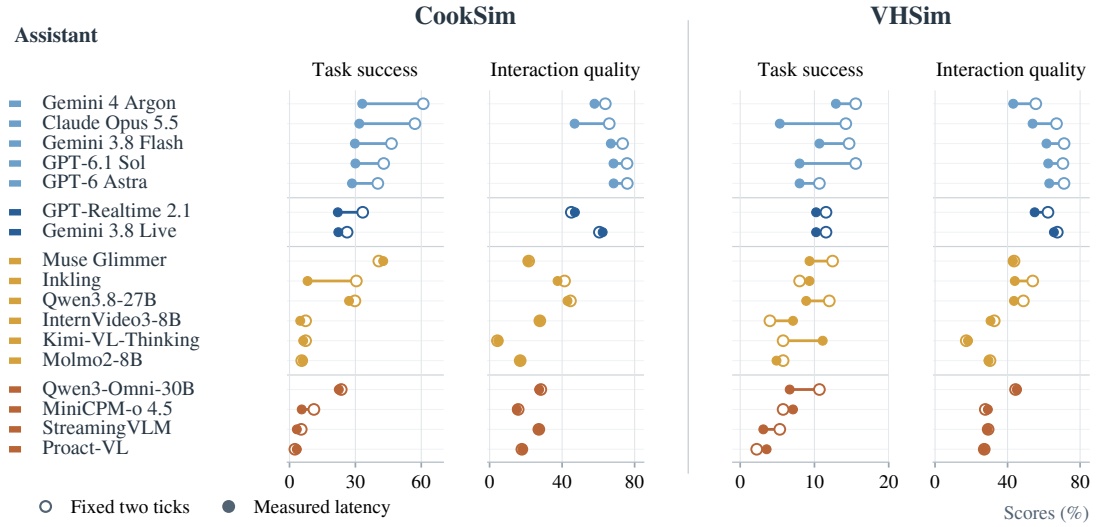


Figure 4: **Effect of assistant latency on task success and interaction quality.** Open markers show the fixed two-tick setting; filled markers show measured latency. Lines connect the same model under the two settings. Scores are averaged across three independent runs.

Interaction quality reflects human preferences. To test the validity of our interaction-quality metric, we compare each assistant's mean score with participants' rankings. The metric agrees with human preferences in **78.5%** of participant–assistant-pair comparisons, corresponding to a pooled Kendall's τ of **+0.57**. This correspondence supports its use as a measure of the assistance experienced by users.

Axis	Measure	Similarity
Communication	Utterance length	0.89
	Speech rate	0.05
	Responsiveness	0.05
Skill	Task success	0.56
Trust	Suggestion uptake	0.77
	Disagreement	0.88
Interruption	Guidance requests	0.98
	Ignored interruptions	0.82
Speed	Action efficiency	0.82
	Parallelism	0.00

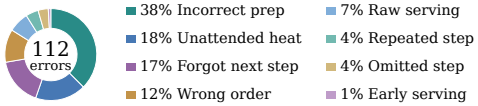


Figure 5: **Behavior comparison of real human and simulated human agent.** See Tab. 13 in the appendix for measure definitions.

Simulated users reproduce most of human behaviors. We compare human and simulated episodes along three dimensions: communication behavior, persona-dependent variation, and action error types (Fig. 5). We use binary rubrics to measure behavioral dimensions and compute the similarity between human and simulated users on each dimension (detailed in appendix). Simulated users reproduce human behavioral differences across most persona axes, with particularly close agreement for trust and interruption preferences. The engine also covers the error types observed in participants' action behaviors, including incorrect preparation, omitted or repeated steps, and assembly-order errors. Action parallelism match less well, possibly because the human speed groups show almost no difference in parallel cooking. We also found a clear deviation between the communication hebavior between human agent and real human: although their utterance lengths are similar, simulated users speak to assistants much more often than human participants. One possible reason is that the simulated user is invoked very frequently to reconsider whether to speak, repeatedly making conversation salient.

4.3 Interactive Visual Gym as an RL environment

We train Qwen3.5-4B with GRPO on CookSim, using Gemini 3.7/3.8 Flash as a classic-novice user. We split CookSim's 150 scenarios into 120 for training and 30 for validation. Each update uses 24 episodes: 6 scenarios with 4 episodes each. Because binary in-time success is sparse, we use the shaped reward $r = \mathbb{I}[\text{success}] + 0.30\,\mathbb{I}[\text{error prevented}] - 0.02\min(N_{\text{false flags}}, 15) - 0.05\min(T/200, 1)$. Here, T counts interaction turns. Task success dominates ($r \geq 0.65$ for successes; $r \leq 0.30$ for completed failures); shaping encourages timely intervention and discourages false flags and excess turns (Appendix C).

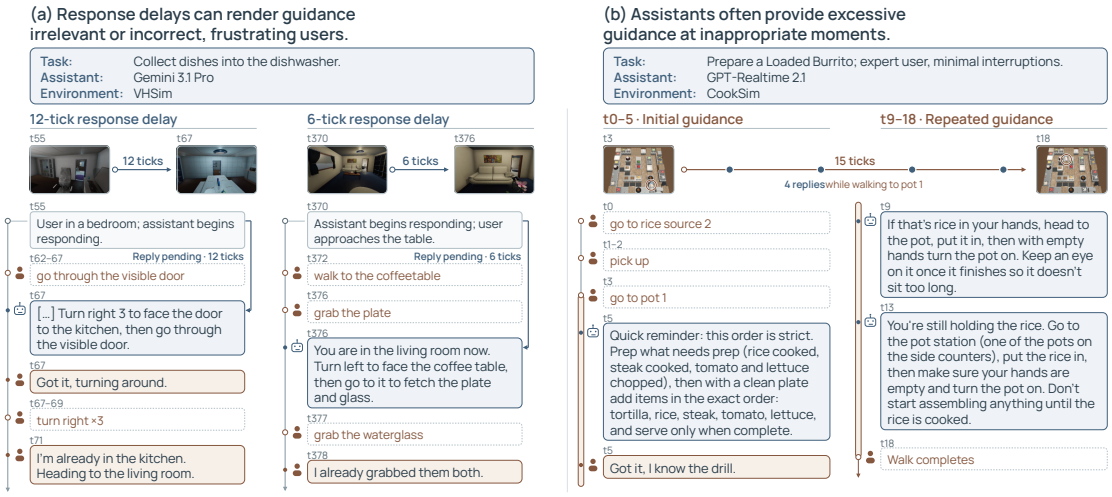


Figure 6: **Qualitative examples of assistant interaction trajectories.** It illustrates recurring problems across assistant types that our dynamic evaluation expose.

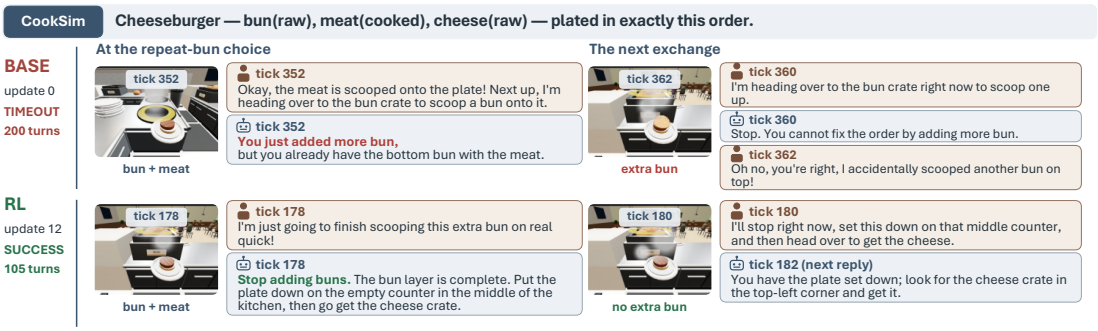


Figure 7: **Qualitative example from RL.** The base assistant misperceives the task state and fails to prevent the error. After RL, the assistant gives timely guidance that prevents the error.

User	Assistant	In-time success	Det. recall	Interaction quality
Gemini 3.7 Flash	Qwen3.5-4B	33.3 ± 3.3	$\mathbf{49.0} \pm 11.9$	29.6 ± 4.8
	+ RL	$\mathbf{42.2} \pm 6.9$	34.9 ± 6.7	$\mathbf{31.1} \pm 0.4$
Gemini 3.8 Flash	Qwen3.5-4B	64.4 ± 6.9	$\mathbf{43.0} \pm 14.5$	24.3 ± 2.1
	+ RL	$\mathbf{67.8} \pm 7.7$	28.7 ± 3.4	$\mathbf{30.2} \pm 0.3$

Table 5: **CookSim validation results before and after RL, by simulated user model.** + RL denotes 12 updates. Scores are percentages, reported as mean ± SD. Bold marks higher means within pairs.

Across three evaluation passes on CookSim's held-out development-validation scenarios with a Gemini 3.7 Flash user, mean in-time task success rate rises from 33.3% to 42.2% after 12 updates, an increase of 8.9 percentage points (Tab. 5). A separate experiment with a Gemini 3.8 Flash user improves mean in-time task success rate by 3.3 percentage points (64.4% to 67.8%) and mean interaction quality by 5.9 points (24.3% to 30.2%). Mean detection recall declines in both settings, possibly because the reward favors selective flagging that can miss uncertain errors, consistent with false flags falling from 517 to 391 across 90 episodes with a Gemini 3.8 Flash user. Fig. 7 illustrates one case where the trained assistant gives timely guidance that prevents an error. These fixed-scenario results show that Interactive Visual Gym supplies a useful learning signal for interactive assistance, paving the foundation for future work to demonstrate generalization across tasks, user models, or domains.

5 Conclusion

We present Interactive Visual Gym, a framework for closed-loop evaluation and training of visual interaction models through joint simulation of task environments and human users. Across three domains, we find substantial limitations in current VLM to serve as reliable assistants. We also validated our interaction paradigm with human study. Finally, a preliminary reinforcement-learning experiment shows that the environments can improve assistant performance.

Acknowledgments

This work was supported in part by Thinking Machines Lab's Tinker Research Grant.

AI use statement

We used generative AI tools, including coding agents, to assist with implementing our simulators and evaluation pipeline, generating synthetic benchmark cases, refining experiments, and interpreting results. We also used them for literature search, figure creation, code editing, and drafting and revising manuscript text. We reviewed AI-assisted outputs and take responsibility for the final code, data, figures, and claims.

Ethics statement

Our human study took place in a browser-based cooking simulator; participants did not perform physical cooking tasks. The study recorded action traces, text exchanges, and preferences, which we analyze and report in aggregate. Because free-text exchanges may contain personal information, any release of raw traces must follow the study's consent and privacy procedures. Performance in our simulators should not be taken as evidence that an assistant is safe for real-world use.

Reproducibility statement

We describe environment and benchmark construction in Appendix G, assistant settings in Appendix H, user prompts in Appendix J, and event placement and scoring in Appendices D, E and I. The appendix also reports the human-study protocol (Appendix F), variation across independent runs (Appendix B), and the reinforcement-learning configuration (Appendix C).

References

Gati V Aher, Rosa I. Arriaga, and Adam Tauman Kalai. Using large language models to simulate multiple humans and replicate human subject studies. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 337–371. PMLR, 23–29 Jul 2023. URL https://proceedings.mlr.press/v202/aher23a.html.

Saleema Amershi, Maya Cakmak, W. Bradley Knox, and Todd Kulesza. Power to the people: The role of humans in interactive machine learning. *AI Magazine*, 35(4):105–120, 2014. doi: 10.1609/aimag.v35i4.2513.

Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. Guidelines for human-ai interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, CHI '19, pp. 1–13. ACM, May 2019. doi: 10.1145/3290605.3300233. URL http://dx.doi.org/10.1145/3290605.3300233.

Anthropic. Claude Opus 5.5, September 2026. URL https://www.anthropic.com/claude-opus-5-5. Model announcement, September 22, 2026.

Apple. iphone user guide. Apple Support, iOS 26 edition, 2026. URL https://support.apple.com/guide/iphone/welcome/26/ios/26.

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2-bench: Evaluating conversational agents in a Dual-Control environment. *arXiv preprint arXiv:2506.07982*, 2025.

Apratim Bhattacharyya, Shweta Mahajan, Sanjay Haresh, Rajeev Yasarla, Reza Pourreza, Litian Liu, Risheek Garrepalli, and Roland Memisevic. Streaming interventions: Can video large language models correct mistakes as they occur? *arXiv preprint arXiv:2606.09547*, 2026.

Matthew Chang, Gunjan Chhablani, Alexander Clegg, Mikael Dallaire Cote, Ruta Desai, Michal Hlavac, Vladimir Karashchuk, Jacob Krantz, Roozbeh Mottaghi, Priyam Parashar, Siddharth Patki, Ishita Prasad, Xavier Puig, Akshara Rai, Ram Ramrakhya, Daniel Tran, Joanne Truong, John M. Turner, Eric Undersander, and Tsung-Yen Yang. PARTNR: A benchmark for planning and reasoning in embodied multi-agent tasks. In *ICLR*, 2025.

Dibyadip Chatterjee, Zhanzhong Pang, Fadime Sener, Yale Song, and Angela Yao. SPOT-Bench: Don't pause! every prediction matters in a streaming video. *arXiv preprint arXiv:2604.24317*, 2026. doi: 10.48550/arXiv.2604.24317. URL https://arxiv.org/abs/2604.24317.

Christopher Clark, Jieyu Zhang, Zixian Ma, Jae Sung Park, Mohammadreza Salehi, Rohun Tripathi, Sangho Lee, Zhongzheng Ren, Chris Dongjoo Kim, Yinuo Yang, Vincent Shao, Yue Yang, Weikai Huang, Ziqi Gao, Taira Anderson, Jianrui Zhang, Jitesh Jain, George Stoica, Winson Han, Ali Farhadi, and Ranjay Krishna. Molmo2: Open weights and data for vision-language models with video understanding and grounding. *arXiv preprint arXiv:2601.10611*, 2026. URL https://arxiv.org/abs/2601.10611.

Junbo Cui, Bokai Xu, Chongyi Wang, Tianyu Yu, Weiyue Sun, Yingjing Xu, Tianran Wang, Zhihui He, Wenshuo Ma, Tianchi Cai, Jiancheng Gui, Luoyuan Zhang, Xian Sun, Fuwei Huang, Moye Chen, Zhuo Lin, Hanyu Liu, Qingxin Gui, Qingzhe Han, Yuyang Wen, Huiping Liu, Rongkang Wang, Yaqi Zhang, Hongliang Wei, Chi Chen, You Li, Kechen Fang, Jie Zhou, Yuxuan Li, Guoyang Zeng, Chaojun Xiao, Yankai Lin, Xu Han, Maosong Sun, Zhiyuan Liu, and Yuan Yao. MiniCPM-o 4.5: Towards real-time full-duplex omni-modal interaction. *arXiv preprint arXiv:2604.27393*, 2026. doi: 10.48550/arXiv.2604.27393. URL https://arxiv.org/abs/2604.27393.

Jim Fan, Guanzhi Wang, Yunfan Jiang, Ajay Mandlekar, Yuncong Yang, Haoyi Zhu, Andrew Tang, De-An Huang, Yuke Zhu, and Anima Anandkumar. MineDojo: Building open-ended embodied agents with internet-scale knowledge. In *Advances in Neural Information Processing Systems*, 2022. URL https://minedojo.org/.

Ghost Town Games. Overcooked! Video game, published by Team17, 2016. URL https://www.team17.com/games/overcooked.

Google. Gemini 3.1 Pro Preview. Gemini API documentation, 2026a. URL https://ai.google.dev/gemini-api/docs/models/gemini-3.1-pro-preview.

Google. Gemini 3.7 Flash. Gemini API documentation, 2026b. URL https://ai.google.dev/gemini-api/docs/models/gemini-3.7-flash.

Google. Gemini 3.8 Flash. Gemini API documentation, 2026c. URL https://ai.google.dev/gemini-api/docs/models/gemini-3.8-flash/.

Google. Gemini 3.8 Live. Gemini API documentation, 2026d. URL https://ai.google.dev/gemini-api/docs/models/gemini-3.8-live.

Zhenpeng Huang, Xinhao Li, Jiaqi Li, Jing Wang, Xiangyu Zeng, Cheng Liang, Tao Wu, Xi Chen, Liang Li, and Limin Wang. Online video understanding: Ovbench and videochat-online. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3328–3338, June 2025.

Koray Kavukcuoglu. Gemini 4 Argon: Our next era of frontier intelligence. Google Blog, September 2026. URL https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-4-argon/.

M. G. Kendall. A new measure of rank correlation. *Biometrika*, 30(1–2):81–93, 1938. doi: 10.1093/biomet/30.1-2.81.

Kimi Team, Angang Du, Bohong Yin, Bowei Xing, Bowen Qu, Bowen Wang, Cheng Chen, Chenlin Zhang, Chenzhuang Du, Chu Wei, Congcong Wang, Dehao Zhang, Dikang Du, Dongliang Wang, Enming Yuan, Enzhe Lu, Fang Li, Flood Sung, Guangda Wei, Guokun Lai, Han Zhu, Hao Ding, Hao Hu, Hao Yang, Hao Zhang, Haoning Wu, Haotian Yao, Haoyu Lu, Heng Wang, Hongcheng Gao, Huabin Zheng, Jiaming Li, Jianlin Su, Jianzhou Wang, Jiaqi Deng, Jiezhong Qiu, Jin Xie, Jinhong Wang, Jingyuan Liu, Junjie Yan, Kun Ouyang, Liang Chen, Lin Sui, Longhui Yu, Mengfan Dong, Mengnan Dong, Nuo Xu, Pengyu Cheng, Qizheng Gu, Runjie Zhou, Shaowei Liu, Sihan Cao, Tao Yu, Tianhui Song, Tongtong Bai, Wei Song, Weiran He, Weixiao Huang, Weixin Xu, Xiaokun Yuan, Xingcheng Yao, Xingzhe Wu, Xinhao Li, Xinxing Zu, Xinyu Zhou, Xinyuan Wang, Y. Charles, Yan Zhong, Yang Li, Yangyang Hu, Yanru Chen, Yejie Wang, Yibo Liu, Yibo Miao, Yidao Qin, Yimin Chen, Yiping Bao, Yiqin Wang, Yongsheng Kang, Yuanxin Liu, Yuhao Dong, Yulun Du, Yuxin Wu, Yuzhi Wang, Yuzi Yan, Zaida Zhou, Zhaowei Li, Zhejun Jiang, Zheng Zhang, Zhilin Yang, Zhiqi Huang, Zihao Huang, Zijia Zhao, Ziwei Chen, and Zongyu Lin. Kimi-VL technical report. *arXiv preprint arXiv:2504.07491*, 2025. URL https://arxiv.org/abs/2504.07491.

Kaustav Kundu, Ritvik Shrivastava, Maxim Arap, Nanshu Wang, Xianhui Zhu, Quintin Fettes, Gautam Tiwari, Parth Suresh, Théo Moutakanni, Alejandro Castillejo Munoz, Allen Bolourchi, Pascale Fung, Pinar Donmez, Babak Damavandi, Anuj Kumar, and Seungwhan Moon. Plan, watch, recover: A benchmark and architectures for proactive procedural assistance. *arXiv preprint arXiv:2606.04970*, 2026.

John D. Lee and Katrina A. See. Trust in automation: Designing for appropriate reliance. *Hum. Factors*, 46(1):50–80, 2004. doi: 10.1518/HFES.46.1.50.30392.

Yukyung Lee, JoongHoon Kim, Jaehee Kim, Hyowon Cho, Jaewook Kang, Pilsung Kang, and Najoung Kim. Checkeval: A reliable llm-as-a-judge framework for evaluating text generation using checklists. In *EMNLP*, pp. 15771–15798, 2025. doi: 10.18653/V1/2025.EMNLP-MAIN.796.

Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabrael Levine, Michael Lingelbach, Jiankai Sun, Mona Anvari, Minjune Hwang, Manasi Sharma, Arman Aydin, Dhruva Bansal, Samuel Hunter, Kyu-Young Kim, Alan Lou, Caleb R. Matthews, Ivan Villa-Renteria, Jerry Huayang Tang, Claire Tang, Fei Xia, Silvio Savarese, Hyowon Gweon, C. Karen Liu, Jiajun Wu, and Li Fei-Fei. BEHAVIOR-1K: A benchmark for embodied AI with 1,000 everyday activities and realistic simulation. In *CoRL*, pp. 80–93, 2022.

Junming Lin, Zheng Fang, Chi Chen, Haoxuan Cheng, Zihao Wan, Fuwen Luo, Ziyue Wang, Peng Li, Yang Liu, and Maosong Sun. StreamingBench: Assessing the gap for MLLMs to achieve streaming video understanding. In *IEEE International Conference on Acoustics, Speech and Signal Processing*, pp. 12147–12151, 2026. doi: 10.1109/ICASSP55912.2026.11463959.

Fang Liu, Jinpeng Chen, Ke Xu, Yuhao Liu, Huankang Guan, Xudong Lu, Bo Yang, Gerhard Hancke, Rui Liu, and Rynson W. H. Lau. GuideMe: Multi-Domain task guidance and intervention in streaming video. In *European Conference on Computer Vision*, 2026. URL https://arxiv.org/abs/2607.02991.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-eval: NLG evaluation using gpt-4 with better human alignment. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 2511–2522, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.153. URL https://aclanthology.org/2023.emnlp-main.153/.

Daniel C. McFarlane. Comparison of four primary methods for coordinating the interruption of people in human-computer interaction. *Human–Computer Interaction*, 17(1):63–139, Mar 2002. ISSN 1532-7051. doi: 10.1207/s15327051hci1701_2. URL http://dx.doi.org/10.1207/S15327051HCI1701_2.

Shikib Mehri and Maxine Eskenazi. Unsupervised evaluation of interactive dialog with DialoGPT. In Olivier Pietquin, Smaranda Muresan, Vivian Chen, Casey Kennington, David Vandyke, Nina Dethlefs, Koji Inoue, Erik Ekstedt, and Stefan Ultes (eds.), *Proceedings of the 21th Annual Meeting of the Special Interest Group on Discourse and Dialogue*, pp. 225–235, 1st virtual meeting, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.sigdial-1.28. URL https://aclanthology.org/2020.sigdial-1.28/.

Fanqing Meng, Lingxiao Du, Jiawei Gu, Jiaqi Liao, Linjie Li, Zijian Wu, Xiangyan Liu, Ziqi Zhao, Mengkang Hu, Zichen Liu, Jiaheng Zhang, and Michael Qizhe Shieh. Gym-V: A unified vision environment system for agentic vision research. *arXiv preprint arXiv:2603.15432*, 2026. doi: 10.48550/arXiv.2603.15432. URL https://arxiv.org/abs/2603.15432.

Meta. Muse-Glimmer-30B model card. Hugging Face model card, 2026. URL https://huggingface.co/meta-models/Muse-Glimmer-30B.

Junbo Niu, Yifei Li, Ziyang Miao, Chunjiang Ge, Yuanhang Zhou, Qihao He, Xiaoyi Dong, Haodong Duan, Shuangrui Ding, Rui Qian, Pan Zhang, Yuhang Zang, Yuhang Cao, Conghui He, and Jiaqi Wang. Ovo-bench: How far is your video-llms from real-world online video understanding? In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 18902–18913, June 2025.

OpenAI. GPT-6 Astra model. OpenAI API documentation, 2026a. URL https://developers.openai.com/api/docs/models/gpt-6-astra.

OpenAI. GPT-Realtime-2.1 model. OpenAI API documentation, 2026b. URL https://developers.openai.com/api/docs/models/gpt-realtime-2.1.

OpenAI. Introducing GPT-6.1 Sol. OpenAI, 2026c. URL https://openai.com/index/introducing-gpt-6-1-sol/.

Orantqing, Shengpeng Ji, Junlong Tong, Jialong Zuo, Dongjie Fu, Di Cao, Yangzhuo Li, Shangda Wu, Franz Evan Theron Veyra, Changhao Pan, Jingyu Lu, Dongchao Yang, Zhifei Xie, Yang Tan, Xiaoyu Shen, Xiaoda Yang, Wenfu Wang, Teddy Sun, Steve Yves, and Zhou Zhao. Multimodal duplex interaction agent. *arXiv preprint arXiv:2609.08977*, 2026. doi: 10.48550/arXiv.2609.08977. URL https://arxiv.org/abs/2609.08977.

Aishwarya Padmakumar, Jesse Thomason, Ayush Shrivastava, Patrick Lange, Anjali Narayan-Chen, Spandana Gella, Robinson Piramuthu, Gökhan Tür, and Dilek Hakkani-Tür. Teach: Task-driven embodied agents that chat. In *AAAI*, pp. 2017–2025, 2022. doi: 10.1609/AAAI.V36I2.20097.

Rohith Peddi, Shivvrat Arya, Bharath Challa, Likhitha Pallapothula, Akshay Vyas, Bhavya Gouripeddi, Qifan Zhang, Jikai Wang, Vasundhara Komaragiri, Eric Ragan, Nicholas Ruozzi, Yu Xiang, and Vibhav Gogate. Captaincook4d: A dataset for understanding errors in procedural activities. In *Advances in Neural Information Processing Systems*, volume 37, pp. 135626–135679, 2024. doi: 10.52202/079017-4307. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/f4a04396c2ed1342a5d8d05e94cb6101-Paper-Datasets_and_Benchmarks_Track.pdf.

Xavier Puig, Kevin Ra, Marko Boben, Jiaman Li, Tingwu Wang, Sanja Fidler, and Antonio Torralba. Virtualhome: Simulating household activities via programs. In *CVPR*, pp. 8494–8502, 2018. doi: 10.1109/CVPR.2018.00886.

Xavier Puig, Eric Undersander, Andrew Szot, Mikael Dallaire Cote, Tsung-Yen Yang, Ruslan Partsey, Ruta Desai, Alexander Clegg, Michal Hlavac, So Yeon Min, Vladimir Vondrus, Théophile Gervet, Vincent-Pierre Berges, John M. Turner, Oleksandr Maksymets, Zsolt Kira, Mrinal Kalakrishnan, Jitendra Malik, Devendra Singh Chaplot, Unnat Jain, Dhruv Batra, Akshara Rai, and Roozbeh Mottaghi. Habitat 3.0: A co-habitat for humans, avatars, and robots. In *ICLR*, 2024.

Qwen Team. Qwen3.5-4B model card. Hugging Face model card, 2026a. URL https://huggingface.co/Qwen/Qwen3.5-4B.

Qwen Team. Qwen3.8-27B model card. Hugging Face model card, 2026b. URL https://huggingface.co/Qwen/Qwen3.8-27B.

Dongchuan Ran, Linyu Ou, Xueheng Li, Wenwen Tong, Chenxu Guo, Hewei Guo, Kaibing Wang, and Lewei Lu. EgoPro-Bench: Benchmarking personalized proactive interaction in egocentric video streams. *arXiv preprint arXiv:2605.07299*, 2026.

Soham Ray, Keshav Dhandhania, Victor Barres, and Karthik Narasimhan. τ-Voice: Benchmarking full-duplex voice agents on real-world domains. *arXiv preprint arXiv:2603.13686*, 2026. doi: 10.48550/arXiv.2603.13686. URL https://arxiv.org/abs/2603.13686.

Jost Schatzmann, Kallirroi Georgila, and Steve Young. Quantitative evaluation of user simulation techniques for spoken dialogue systems. In Laila Dybkjær and Wolfgang Minker (eds.), *Proceedings of the 6th SIGdial Workshop on Discourse and Dialogue*, pp. 45–54, Lisbon, Portugal, September 2005. Special Interest Group on Discourse and Dialogue (SIGdial). doi: 10.18653/v1/2005.sigdial-1.6. URL https://aclanthology.org/2005.sigdial-1.6/.

Preethi Seshadri, Samuel Cahyawijaya, Ayomide Odumakinde, Sameer Singh, and Seraphina Goldfarb-Tarrant. Lost in simulation: Llm-simulated users are unreliable proxies for human users in agentic evaluations. In *ACL*, pp. 47423–47439, 2026. doi: 10.18653/V1/2026.ACL-LONG.2192.

Yijia Shao, Vinay Samuel, Yucheng Jiang, John Yang, and Diyi Yang. Collaborative gym: A framework for enabling and evaluating human-agent collaboration. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/pdf/6a51051cb7c11888730756bf5ac33aab49f65084.pdf.

Ben Shneiderman. Universal usability. *Communications of the ACM*, 43(5):84–91, May 2000. ISSN 1557-7317. doi: 10.1145/332833.332843. URL http://dx.doi.org/10.1145/332833.332843.

Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke Zettlemoyer, and Dieter Fox. ALFRED: A benchmark for interpreting grounded instructions for everyday tasks. In *CVPR*, pp. 10737–10746, 2020. doi: 10.1109/CVPR42600.2020.01075.

Weiwei Sun, Xuhui Zhou, Weihua Du, Xingyao Wang, Sean Welleck, Graham Neubig, Maarten Sap, and Yiming Yang. Training proactive and personalized LLM agents. In *Third Conference on Language Modeling*, 2026. URL https://arxiv.org/abs/2511.02208.

Thinking Machines Lab. Inkling: Our open-weights model, July 2026a. URL https://thinkingmachines.ai/news/introducing-inkling/. Model announcement, July 15, 2026.

Thinking Machines Lab. Interaction models: A scalable approach to human-AI collaboration. *Thinking Machines Lab: Connectionism*, May 2026b. doi: 10.64434/tml.20260511. URL https://thinkingmachines.ai/blog/interaction-models/.

Pavan Kumar Anasosalu Vasu, Cem Koc, Fartash Faghri, Chun-Liang Li, Bo Feng, Zhengfeng Lai, Meng Cao, Oncel Tuzel, and Hadi Pouransari. VSAS-Bench: Real-time evaluation of visual streaming assistant models. *arXiv preprint arXiv:2604.07634*, 2026. doi: 10.48550/arXiv.2604.07634. URL https://arxiv.org/abs/2604.07634.

Mrinal Verghese, Brian Chen, Hamid Eghbalzadeh, Tushar Nagarajan, and Ruta Desai. User-in-the-loop evaluation of multimodal LLMs for activity assistance, 2024. URL https://arxiv.org/abs/2408.03160.

Zirui Wang, Junyi Zhang, Jiaxin Ge, Long Lian, Letian Fu, Lisa Dunlap, Ken Goldberg, Xudong Wang, Ion Stoica, David M. Chan, Sewon Min, and Joseph E. Gonzalez. VisGym: Diverse, customizable, scalable environments for multimodal agents. *arXiv preprint arXiv:2601.16973*, 2026. doi: 10.48550/arXiv.2601.16973. URL https://arxiv.org/abs/2601.16973.

Wayne A. Wickelgren. Speed-accuracy tradeoff and information processing dynamics. *Acta Psychologica*, 41(1):67–85, Feb 1977. ISSN 0001-6918. doi: 10.1016/0001-6918(77)90012-9. URL http://dx.doi.org/10.1016/0001-6918(77)90012-9.

Muzhe Wu, Zuchen Li, Xu Wang, and Anhong Guo. Generative tutorial: Towards live contextualized visual instructions for physical tasks, 2026. URL https://arxiv.org/abs/2609.24955.

Jin Xu, Zhifang Guo, Hangrui Hu, Yunfei Chu, Xiong Wang, Jinzheng He, Yuxuan Wang, Xian Shi, Ting He, Xinfa Zhu, Yuanjun Lv, Yongqi Wang, Dake Guo, He Wang, Linhan Ma, Pei Zhang, Xinyu Zhang, Hongkun Hao, Zishan Guo, Baosong Yang, Bin Zhang, Ziyang Ma, Xipin Wei, Shuai Bai, Keqin Chen, Xuejing Liu, Peng Wang, Mingkun Yang, Dayiheng Liu, Xingzhang Ren, Bo Zheng, Rui Men, Fan Zhou, Bowen Yu, Jianxin Yang, Le Yu, Jingren Zhou, and Junyang Lin. Qwen3-Omni technical report. *arXiv preprint arXiv:2509.17765*, 2025a. URL https://arxiv.org/abs/2509.17765.

Ruyi Xu, Guangxuan Xiao, Yukang Chen, Liuning He, Yao Lu, and Song Han. StreamingVLM: Real-time understanding for infinite video streams. *arXiv preprint arXiv:2510.09608*, 2025b. URL https://arxiv.org/abs/2510.09608.

Weicai Yan, Yuhong Dai, Qi Ran, Haodong Li, Wang Lin, Tao Jin, Xing Xie, Hao Liao, and Jianxun Lian. Proact-VL: A proactive videollm for real-time ai companions. *arXiv preprint arXiv:2603.03447*, 2026a. URL https://arxiv.org/abs/2603.03447.

Ziang Yan, Sheng Xia, Jiashuo Yu, Yue Wu, Tianxiang Jiang, Songze Li, Kanghui Tian, Yicheng Xu, Yinan He, Kai Chen, Limin Wang, Yu Qiao, and Yi Wang. InternVideo3: Agentify foundation models with multimodal contextual reasoning. *arXiv preprint arXiv:2606.12195*, 2026b. URL https://arxiv.org/abs/2606.12195.

Dingyu Yao, Junhao Zhou, Chenxu Yang, Chuanyu Qin, Xiangyu Zeng, Yifei Li, Haowen Hou, Zheming Liang, Congcong Wang, Kaiwen Tuo, Jun Zhang, Yuhan Zhu, Yuhang Cao, Shenglong Ye, Shuai Xie, Shuhuan Gu, Haoyang Huang, Qingyi Si, Nan Duan, and Jiaqi Wang. JoyAI-VL-Interaction: Real-time vision-language interaction intelligence. *arXiv preprint arXiv:2606.14777*, 2026. doi: 10.48550/arXiv.2606.14777. URL https://arxiv.org/abs/2606.14777.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for Tool-Agent-User interaction in Real-World domains. *arXiv preprint arXiv:2406.12045*, 2024.

Jianing Zhang, Chenhao Zheng, Yajun Yang, Max Argus, Rustin Soraki, Winson Han, Taira Anderson, Chun-Liang Li, Shuo Liu, Jiafei Duan, Zhongzheng Ren, Jieyu Zhang, and Ranjay Krishna. Molmomotion: Forecasting point trajectories in 3d with language instruction, 2026. URL https://arxiv.org/abs/2606.18558.

Yichi Zhang, Xin Luna Dong, Zhaojiang Lin, Andrea Madotto, Anuj Kumar, Babak Damavandi, Joyce Chai, and Seungwhan Moon. Proactive assistant dialogue generation from streaming egocentric videos. In *EMNLP*, pp. 12044–12068, 2025a. doi: 10.18653/V1/2025.EMNLP-MAIN.605.

Yichi Zhang, Run Peng, Lingyun Wu, Yinpei Dai, Xuweiyi Chen, Qiaozi Gao, and Joyce Chai. Bootstrapping visual assistant modeling with situated interaction simulation. In *Proceedings of the Second Conference on Language Modeling*, 2025b. URL https://openreview.net/forum?id=S4nTXotasR.

Chenhao Zheng, Ayush Shrivastava, and Andrew Owens. Exif as language: Learning cross-modal associations between images and camera metadata. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 6945–6956, June 2023.

Chenhao Zheng, Jieyu Zhang, Aniruddha Kembhavi, and Ranjay Krishna. Iterated learning improves compositionality in large vision-language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 13785–13795, June 2024.

Chenhao Zheng, Jieyu Zhang, Mohammadreza Salehi, Ziqi Gao, Vishnu Iyengar, Norimasa Kobori, Quan Kong, and Ranjay Krishna. One trajectory, one token: Grounded video tokenization via panoptic sub-object trajectory. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 23156–23166, October 2025.

Chenhao Zheng, Jieyu Zhang, Jianing Zhang, Weikai Huang, Ashutosh Kumar, Quan Kong, Oncel Tuzel, Chun-Liang Li, and Ranjay Krishna. Trajtok: Learning trajectory tokens enables better video understanding, 2026. URL https://arxiv.org/abs/2602.22779.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2307.13854.

Kejian Zhu, Zhuoran Jin, Dongqi Huang, Hongbang Yuan, Yupu Hao, Kang Liu, and Jun Zhao. Beyond simply environment scaling: Designing effective environment distributions for multimodal agent learning. *arXiv preprint arXiv:2608.03571*, 2026. doi: 10.48550/arXiv.2608.03571. URL https://arxiv.org/abs/2608.03571.

APPENDIX CONTENTS

A Sensitivity to the Simulated-User Model 18
B Variation Across Independent Runs 18
C Reinforcement-Learning Setup and Results 22
D Task Success and Error Detection 23
E Temporal Questions and Proactive Requests 23
F Human Study: Protocol and Behavioral Comparisons 25
G Environment and Benchmark Construction 28
H Assistant Settings 30
I Interaction-Quality Rubric and Judging 32
J User Harness and Prompt Design 33
K Qualitative Interaction Trajectories 36

A SENSITIVITY TO THE SIMULATED-USER MODEL

We examine how the choice of base model for the simulated human agent affects evaluation results. We fix the visual interaction model to Gemini 3.7 Flash and vary the model controlling the simulated user in CookSim, evaluating on the same subset of 90 benchmark episodes (Tab. 6).

User model	In-time success (%)	Any success (%)	Words/episode
Gemini 3.1 Pro	28.9	56.7	228.6
GPT-6 Astra	23.3	48.9	215.2
Gemini 3.7 Flash	21.1	52.2	147.1
Qwen3.8-27B	2.2	27.8	423.7

Table 6: **Sensitivity to the simulated-user model.** Any success ignores the completion deadline. Word counts refer to the simulated user, not the assistant.

We find that GPT-6 Astra and Gemini 3.7 Flash yield similar task-success rates when used to evaluate the same assistant, whereas Gemini 3.1 Pro differ a bit. Local open-weight models yield much lower task-success rates, potentially reflecting smaller models' difficulty following the complex instructions required to elicit the intended user behavior.

B VARIATION ACROSS INDEPENDENT RUNS

We examine the variability of closed-loop evaluation by running each assistant three times with the same tasks, personas, and scheduled events. Tabs. 7 to 9 report in-time success, detection recall, and interaction quality for each run, together with their mean and population standard deviation (SD). We evaluate CookSim and VHSim under both fixed two-tick and measured assistant delay. All scores are percentages.

	In-time success (%)		Detection recall (%)		Interaction quality (%)	
Assistant	R1 / R2 / R3	Mean ± SD	R1 / R2 / R3	Mean ± SD	R1 / R2 / R3	Mean ± SD
Fixed two-tick delay						
Gemini 4 Argon	64.0 / 58.7 / 60.0	60.9 ± 2.3	81.1 / 80.6 / 79.0	80.2 ± 0.9	63.5 / 64.5 / 63.5	63.8 ± 0.5
Claude Opus 5.5	59.3 / 60.0 / 52.0	57.1 ± 3.6	87.0 / 87.0 / 83.9	86.0 ± 1.5	65.1 / 66.8 / 66.0	66.0 ± 0.7
Gemini 3.1 Pro	46.0 / 51.3 / 47.3	48.2 ± 2.3	83.4 / 87.0 / 84.8	85.1 ± 1.5	36.0 / 37.1 / 38.2	37.1 ± 0.9
Gemini 3.8 Flash	48.0 / 46.0 / 45.3	46.4 ± 1.1	77.7 / 77.7 / 78.3	77.9 ± 0.3	73.7 / 72.8 / 73.6	73.4 ± 0.4
GPT-6.1 Sol	45.3 / 42.7 / 40.7	42.9 ± 1.9	79.6 / 80.6 / 79.6	79.9 ± 0.5	76.2 / 75.1 / 76.1	75.8 ± 0.5
GPT-6 Astra	42.0 / 38.7 / 40.0	40.2 ± 1.4	80.6 / 80.4 / 80.1	80.4 ± 0.2	77.2 / 75.5 / 75.2	76.0 ± 0.9
GPT-Realtime 2.1	34.0 / 32.0 / 34.0	33.3 ± 0.9	70.0 / 68.1 / 66.0	68.0 ± 1.6	43.8 / 45.8 / 45.6	45.1 ± 0.9
Gemini 3.8 Live	22.7 / 28.0 / 28.0	26.2 ± 2.5	56.1 / 60.3 / 59.0	58.5 ± 1.7	60.2 / 62.1 / 59.4	60.6 ± 1.2
Muse Glimmer	44.7 / 42.0 / 35.3	40.7 ± 3.9	61.3 / 63.2 / 57.0	60.5 ± 2.6	21.2 / 22.1 / 21.3	21.6 ± 0.4
Inkling	28.7 / 31.3 / 31.3	30.4 ± 1.3	79.8 / 78.2 / 80.5	79.5 ± 0.9	41.6 / 41.1 / 41.3	41.3 ± 0.2
Qwen3.8-27B	36.7 / 26.0 / 26.7	29.8 ± 4.9	62.7 / 61.4 / 57.7	60.6 ± 2.1	44.3 / 44.2 / 45.4	44.6 ± 0.5
InternVideo3-8B	9.3 / 5.3 / 7.3	7.3 ± 1.6	23.0 / 25.8 / 28.0	25.6 ± 2.1	26.3 / 28.3 / 28.5	27.7 ± 1.0
Kimi-VL-Thinking	9.3 / 6.7 / 6.0	7.3 ± 1.4	55.6 / 56.1 / 58.1	56.6 ± 1.1	4.4 / 4.3 / 4.2	4.3 ± 0.0
Molmo2-8B	6.0 / 4.0 / 6.7	5.6 ± 1.1	5.4 / 3.9 / 5.9	5.1 ± 0.9	17.0 / 16.8 / 16.8	16.9 ± 0.1
Qwen3-Omni-30B	27.3 / 25.3 / 18.0	23.6 ± 4.0	39.9 / 40.2 / 36.8	39.0 ± 1.5	27.6 / 28.8 / 28.4	28.3 ± 0.5
MiniCPM-o 4.5	10.0 / 10.7 / 12.7	11.1 ± 1.1	39.0 / 37.6 / 42.2	39.6 ± 1.9	15.6 / 15.7 / 16.0	15.8 ± 0.1
StreamingVLM	5.3 / 6.0 / 4.7	5.3 ± 0.5	1.1 / 0.3 / 0.6	0.7 ± 0.3	27.2 / 26.6 / 27.6	27.2 ± 0.4
Proact-VL	2.7 / 2.0 / 2.7	2.4 ± 0.3	0.5 / 0.6 / 1.2	0.8 ± 0.3	18.1 / 17.0 / 18.2	17.8 ± 0.5
Measured assistant delay						
Gemini 4 Argon	35.3 / 31.3 / 32.7	33.1 ± 1.7	69.9 / 65.2 / 65.6	66.9 ± 2.1	57.6 / 57.6 / 58.3	57.9 ± 0.3
Claude Opus 5.5	25.3 / 32.0 / 38.0	31.8 ± 5.2	62.4 / 64.1 / 66.2	64.2 ± 1.6	47.1 / 48.7 / 44.9	46.9 ± 1.6
Gemini 3.1 Pro	21.3 / 22.0 / 18.0	20.4 ± 1.7	53.3 / 48.3 / 44.5	48.7 ± 3.6	33.8 / 36.0 / 34.5	34.8 ± 0.9
Gemini 3.8 Flash	28.7 / 32.0 / 28.7	29.8 ± 1.6	60.2 / 61.8 / 62.0	61.3 ± 0.8	66.5 / 66.1 / 68.0	66.9 ± 0.8
GPT-6.1 Sol	28.7 / 32.7 / 28.7	30.0 ± 1.9	68.2 / 68.5 / 65.6	67.5 ± 1.3	69.5 / 67.3 / 68.3	68.4 ± 0.9
GPT-6 Astra	28.0 / 25.3 / 32.0	28.4 ± 2.7	63.4 / 60.9 / 66.7	63.6 ± 2.4	68.2 / 66.0 / 71.1	68.4 ± 2.1
GPT-Realtime 2.1	17.3 / 22.7 / 26.0	22.0 ± 3.6	56.0 / 62.4 / 63.5	60.6 ± 3.3	46.5 / 47.2 / 47.5	47.1 ± 0.4
Gemini 3.8 Live	18.7 / 22.0 / 26.0	22.2 ± 3.0	52.4 / 53.2 / 53.6	53.0 ± 0.5	63.4 / 62.2 / 61.6	62.4 ± 0.7
Muse Glimmer	43.3 / 39.3 / 45.3	42.7 ± 2.5	63.0 / 57.6 / 61.5	60.7 ± 2.3	21.2 / 21.1 / 21.4	21.2 ± 0.1
Inkling	5.3 / 8.7 / 10.7	8.2 ± 2.2	37.9 / 37.1 / 39.3	38.1 ± 0.9	38.4 / 37.9 / 36.2	37.5 ± 0.9
Qwen3.8-27B	32.7 / 24.0 / 24.7	27.1 ± 3.9	55.8 / 58.3 / 57.1	57.1 ± 1.0	43.1 / 42.0 / 43.8	43.0 ± 0.7
InternVideo3-8B	4.7 / 4.0 / 6.0	4.9 ± 0.8	15.5 / 15.7 / 13.0	14.7 ± 1.2	27.7 / 28.1 / 27.1	27.7 ± 0.4
Kimi-VL-Thinking	6.0 / 6.0 / 6.7	6.2 ± 0.3	48.2 / 45.8 / 44.4	46.1 ± 1.5	4.6 / 4.9 / 4.8	4.8 ± 0.2
Molmo2-8B	6.7 / 4.0 / 7.3	6.0 ± 1.4	3.7 / 5.4 / 4.4	4.5 ± 0.7	17.8 / 15.4 / 17.2	16.8 ± 1.0
Qwen3-Omni-30B	24.0 / 24.7 / 18.7	22.4 ± 2.7	40.4 / 45.4 / 41.6	42.5 ± 2.1	28.9 / 26.2 / 27.0	27.3 ± 1.1
MiniCPM-o 4.5	6.7 / 4.7 / 5.3	5.6 ± 0.8	39.3 / 36.2 / 33.3	36.3 ± 2.4	15.5 / 15.1 / 14.7	15.1 ± 0.3
StreamingVLM	4.7 / 2.0 / 3.3	3.3 ± 1.1	1.4 / 0.8 / 0.8	1.0 ± 0.3	26.9 / 27.7 / 26.9	27.2 ± 0.4
Proact-VL	4.0 / 3.3 / 2.7	3.3 ± 0.5	1.4 / 0.9 / 0.8	1.0 ± 0.3	18.1 / 17.7 / 18.5	18.1 ± 0.3

Table 7: **Independent-run results in CookSim.** Scores are percentages for three independent runs (R1–R3), with their mean and population standard deviation (SD).

	In-time success (%)		Detection recall (%)		Interaction quality (%)	
Assistant	R1 / R2 / R3	Mean ± SD	R1 / R2 / R3	Mean ± SD	R1 / R2 / R3	Mean ± SD
Fixed two-tick delay						
Gemini 4 Argon	20.0 / 13.3 / 13.3	15.6 ± 3.1	81.4 / 79.8 / 82.3	81.2 ± 1.0	57.5 / 54.8 / 54.4	55.6 ± 1.4
Claude Opus 5.5	14.7 / 13.3 / 14.7	14.2 ± 0.6	63.9 / 64.9 / 59.1	62.6 ± 2.5	66.2 / 68.2 / 66.6	67.0 ± 0.9
Gemini 3.1 Pro	17.3 / 12.0 / 14.7	14.7 ± 2.2	89.6 / 90.1 / 89.3	89.6 ± 0.3	53.8 / 56.4 / 54.8	55.0 ± 1.1
Gemini 3.8 Flash	16.0 / 17.3 / 10.7	14.7 ± 2.9	73.5 / 73.7 / 69.4	72.2 ± 2.0	69.5 / 71.4 / 72.6	71.2 ± 1.3
GPT-6.1 Sol	17.3 / 16.0 / 13.3	15.6 ± 1.7	56.7 / 64.1 / 60.0	60.3 ± 3.0	70.0 / 70.8 / 70.8	70.5 ± 0.4
GPT-6 Astra	10.7 / 9.3 / 12.0	10.7 ± 1.1	63.6 / 60.4 / 62.3	62.1 ± 1.3	72.1 / 70.0 / 71.3	71.1 ± 0.9
GPT-Realtime 2.1	9.3 / 12.0 / 13.3	11.6 ± 1.7	58.5 / 56.6 / 67.3	60.8 ± 4.7	63.2 / 61.2 / 62.4	62.3 ± 0.8
Gemini 3.8 Live	12.0 / 14.7 / 8.0	11.6 ± 2.7	18.6 / 25.5 / 29.8	24.6 ± 4.6	68.3 / 66.0 / 68.1	67.4 ± 1.0
Muse Glimmer	12.0 / 14.7 / 10.7	12.4 ± 1.7	17.5 / 14.3 / 11.1	14.3 ± 2.6	43.2 / 42.9 / 44.6	43.6 ± 0.7
Inkling	9.3 / 9.3 / 5.3	8.0 ± 1.9	78.6 / 81.1 / 77.4	79.0 ± 1.5	54.3 / 54.4 / 53.0	53.9 ± 0.6
Qwen3.8-27B	10.7 / 14.7 / 10.7	12.0 ± 1.9	31.6 / 34.3 / 29.4	31.8 ± 2.0	48.8 / 48.4 / 48.9	48.7 ± 0.2
InternVideo3-8B	2.7 / 5.3 / 4.0	4.0 ± 1.1	0.0 / 0.0 / 0.0	0.0 ± 0.0	30.1 / 35.2 / 32.6	32.6 ± 2.1
Kimi-VL-Thinking	5.3 / 8.0 / 4.0	5.8 ± 1.7	64.9 / 83.5 / 73.6	74.0 ± 7.6	17.6 / 16.4 / 17.7	17.2 ± 0.6
Molmo2-8B	6.7 / 5.3 / 5.3	5.8 ± 0.6	2.1 / 0.0 / 1.1	1.1 ± 0.9	31.9 / 27.6 / 31.4	30.3 ± 1.9
Qwen3-Omni-30B	12.0 / 10.7 / 9.3	10.7 ± 1.1	3.9 / 3.1 / 1.1	2.7 ± 1.2	44.5 / 44.2 / 44.4	44.4 ± 0.1
MiniCPM-o 4.5	5.3 / 8.0 / 4.0	5.8 ± 1.7	6.2 / 11.2 / 11.8	9.8 ± 2.5	27.0 / 26.7 / 29.4	27.7 ± 1.2
StreamingVLM	6.7 / 5.3 / 4.0	5.3 ± 1.1	0.0 / 0.0 / 0.0	0.0 ± 0.0	29.5 / 28.7 / 29.7	29.3 ± 0.5
Proact-VL	2.7 / 2.7 / 1.3	2.2 ± 0.6	0.0 / 0.0 / 0.0	0.0 ± 0.0	26.4 / 27.1 / 28.0	27.1 ± 0.7
Measured assistant delay						
Gemini 4 Argon	13.3 / 13.3 / 12.0	12.9 ± 0.6	70.9 / 73.4 / 70.0	71.4 ± 1.4	41.9 / 43.0 / 44.4	43.1 ± 1.0
Claude Opus 5.5	9.3 / 1.3 / 5.3	5.3 ± 3.3	59.8 / 53.6 / 51.4	54.9 ± 3.6	54.1 / 55.0 / 52.2	53.8 ± 1.2
Gemini 3.1 Pro	9.3 / 2.7 / 6.7	6.2 ± 2.7	74.4 / 64.3 / 63.6	67.5 ± 4.9	43.9 / 42.6 / 42.0	42.8 ± 0.8
Gemini 3.8 Flash	12.0 / 6.7 / 13.3	10.7 ± 2.9	55.0 / 57.4 / 61.0	57.8 ± 2.5	64.9 / 59.6 / 59.9	61.4 ± 2.4
GPT-6.1 Sol	6.7 / 9.3 / 8.0	8.0 ± 1.1	59.0 / 59.3 / 58.8	59.0 ± 0.2	61.8 / 62.5 / 63.0	62.4 ± 0.5
GPT-6 Astra	8.0 / 9.3 / 6.7	8.0 ± 1.1	46.3 / 45.8 / 51.5	47.9 ± 2.6	62.5 / 62.2 / 64.3	63.0 ± 1.0
GPT-Realtime 2.1	9.3 / 8.0 / 13.3	10.2 ± 2.3	34.5 / 34.1 / 37.9	35.5 ± 1.7	55.8 / 53.2 / 55.8	54.9 ± 1.2
Gemini 3.8 Live	13.3 / 9.3 / 8.0	10.2 ± 2.3	24.1 / 14.5 / 20.8	19.8 ± 4.0	65.3 / 65.5 / 65.9	65.6 ± 0.3
Muse Glimmer	5.3 / 13.3 / 9.3	9.3 ± 3.3	9.3 / 19.8 / 12.4	13.9 ± 4.4	43.3 / 43.0 / 42.6	43.0 ± 0.3
Inkling	9.3 / 8.0 / 10.7	9.3 ± 1.1	51.6 / 45.2 / 52.8	49.8 ± 3.3	44.9 / 45.4 / 41.7	44.0 ± 1.6
Qwen3.8-27B	10.7 / 8.0 / 8.0	8.9 ± 1.3	31.6 / 34.9 / 24.8	30.4 ± 4.2	43.5 / 43.2 / 43.9	43.5 ± 0.3
InternVideo3-8B	6.7 / 6.7 / 8.0	7.1 ± 0.6	0.0 / 0.0 / 0.0	0.0 ± 0.0	32.2 / 28.4 / 31.2	30.6 ± 1.6
Kimi-VL-Thinking	9.3 / 17.3 / 6.7	11.1 ± 4.5	70.7 / 67.0 / 67.3	68.3 ± 1.7	18.6 / 18.3 / 17.7	18.2 ± 0.4
Molmo2-8B	6.7 / 4.0 / 4.0	4.9 ± 1.3	0.0 / 0.0 / 0.0	0.0 ± 0.0	31.8 / 27.7 / 28.9	29.5 ± 1.7
Qwen3-Omni-30B	6.7 / 6.7 / 6.7	6.7 ± 0.0	6.4 / 6.1 / 2.2	4.9 ± 1.9	45.9 / 44.0 / 45.3	45.1 ± 0.8
MiniCPM-o 4.5	8.0 / 5.3 / 8.0	7.1 ± 1.3	15.6 / 8.9 / 9.0	11.2 ± 3.1	29.3 / 29.6 / 28.5	29.1 ± 0.5
StreamingVLM	4.0 / 4.0 / 1.3	3.1 ± 1.3	0.0 / 0.0 / 0.0	0.0 ± 0.0	29.4 / 29.6 / 28.0	29.0 ± 0.7
Proact-VL	6.7 / 4.0 / 0.0	3.6 ± 2.7	0.0 / 0.0 / 0.0	0.0 ± 0.0	27.7 / 27.5 / 25.8	27.0 ± 0.9

Table 8: **Independent-run results in VHSim.** Scores are percentages for three independent runs (R1–R3), with their mean and population standard deviation (SD).

	In-time success (%)		Detection recall (%)		Interaction quality (%)	
Assistant	R1 / R2 / R3	Mean ± SD	R1 / R2 / R3	Mean ± SD	R1 / R2 / R3	Mean ± SD
ScreenSim						
Gemini 4 Argon	82.2 / 86.7 / 82.2	83.7 ± 2.1	100.0 / 100.0 / 100.0	100.0 ± 0.0	84.7 / 87.6 / 89.1	87.1 ± 1.8
Claude Opus 5.5	86.7 / 90.0 / 91.1	89.3 ± 1.9	100.0 / 100.0 / 100.0	100.0 ± 0.0	82.7 / 85.4 / 84.0	84.0 ± 1.1
Gemini 3.1 Pro	85.6 / 84.4 / 88.9	86.3 ± 1.9	99.4 / 100.0 / 99.4	99.6 ± 0.3	85.7 / 83.7 / 85.2	84.9 ± 0.9
Gemini 3.8 Flash	78.9 / 75.6 / 81.1	78.5 ± 2.3	97.6 / 97.6 / 98.3	97.8 ± 0.3	89.4 / 89.5 / 89.3	89.4 ± 0.1
GPT-6.1 Sol	85.6 / 88.9 / 87.8	87.4 ± 1.4	100.0 / 100.0 / 100.0	100.0 ± 0.0	86.7 / 85.7 / 87.2	86.5 ± 0.6
GPT-6 Astra	91.1 / 92.2 / 87.8	90.4 ± 1.9	100.0 / 100.0 / 100.0	100.0 ± 0.0	86.1 / 86.0 / 86.7	86.3 ± 0.3
GPT-Realtime 2.1	64.4 / 67.8 / 67.8	66.7 ± 1.6	100.0 / 100.0 / 95.9	98.6 ± 1.9	84.7 / 85.1 / 83.7	84.5 ± 0.6
Gemini 3.8 Live	68.9 / 70.0 / 76.7	71.9 ± 3.4	95.3 / 96.4 / 96.6	96.1 ± 0.6	88.4 / 89.4 / 89.5	89.1 ± 0.5
Muse Glimmer	43.3 / 46.7 / 42.2	44.1 ± 1.9	95.9 / 94.8 / 97.6	96.1 ± 1.2	55.3 / 50.4 / 55.8	53.8 ± 2.4
Inkling	47.8 / 52.2 / 43.3	47.8 ± 3.6	97.1 / 99.4 / 98.8	98.4 ± 1.0	76.2 / 76.6 / 78.2	77.0 ± 0.8
Qwen3.8-27B	65.6 / 67.8 / 70.0	67.8 ± 1.8	93.8 / 97.7 / 97.2	96.2 ± 1.7	86.5 / 87.1 / 85.0	86.2 ± 0.9
InternVideo3-8B	1.1 / 0.0 / 0.0	0.4 ± 0.5	39.3 / 38.3 / 45.6	41.1 ± 3.2	33.1 / 32.0 / 36.7	33.9 ± 2.0
Kimi-VL-Thinking	11.1 / 7.8 / 10.0	9.6 ± 1.4	82.3 / 78.1 / 79.7	80.0 ± 1.7	32.8 / 32.7 / 33.5	33.0 ± 0.3
Molmo2-8B	2.2 / 0.0 / 0.0	0.7 ± 1.0	16.7 / 24.3 / 23.6	21.5 ± 3.4	38.1 / 39.5 / 40.0	39.2 ± 0.8
Qwen3-Omni-30B	13.3 / 14.4 / 17.8	15.2 ± 1.9	54.8 / 59.1 / 61.0	58.3 ± 2.6	57.4 / 57.6 / 59.5	58.2 ± 0.9
MiniCPM-o 4.5	2.2 / 3.3 / 2.2	2.6 ± 0.5	42.0 / 46.4 / 47.1	45.2 ± 2.2	40.1 / 37.5 / 37.8	38.5 ± 1.1
StreamingVLM	0.0 / 0.0 / 0.0	0.0 ± 0.0	2.7 / 0.0 / 0.0	0.9 ± 1.3	36.4 / 38.5 / 36.5	37.2 ± 1.0
Proact-VL	0.0 / 0.0 / 0.0	0.0 ± 0.0	0.0 / 1.4 / 4.2	1.9 ± 1.7	25.8 / 29.5 / 28.5	27.9 ± 1.6

Table 9: **Independent-run results in ScreenSim.** Scores are percentages for three independent runs (R1–R3), with their mean and population standard deviation (SD).

C Reinforcement-Learning Setup and Results

These proof-of-concept RL experiments are separate from the multi-model benchmark and user-model ablation. The main CookSim validation comparison appears in Tab. 5; Tabs. 10 and 11 summarize additional CookSim and ScreenSim results.

CookSim. The assistant is Qwen3.5-4B, trained with GRPO while freezing the vision encoder. The language model uses Adam with a constant learning rate of 5×10^{-7}, zero weight decay, and no KL or entropy regularization. Generation uses temperature 0.8, top-$p = 1.0$, a maximum of 512 output tokens, and a 16,384-token context window. Gemini 3.7 Flash simulates a classic-novice user.

The training set contains 120 scenarios specifying a recipe, layout, and scheduled user error. An update samples six scenario groups with five episodes each and retains the first four valid completions per group, yielding 24 training episodes. The reward is

$$\begin{aligned} r = {} & \mathbf{1}[\text{success}] + 0.30\,\mathbf{1}[\text{error prevented}] \\ & - 0.02\,\min(N_{\text{false flags}}, 15) - 0.05\,\min(T/200, 1) \end{aligned} \tag{3}$$

Here, T is the number of interaction turns.

CookSim evaluation. The original evaluation uses three stochastic passes over 120 training and 30 held-out development-validation scenarios. Validation variants share recipe/layout cells with training; there is no separate frozen test set. Success is CookSim's native `won` outcome. A separate three-pass evaluation covers all 30 validation scenarios with complete traces; requests for Gemini 3.7 Flash were routed to effective Gemini 3.8 Flash, as labeled in Tab. 5. Means and sample SDs reflect rollout variation on fixed scenarios. The two evaluations differ in rollouts, effective user model, and interaction-quality judging, so their absolute scores are not directly comparable.

The original interaction-quality scores use Gemini judgments on matched subsets of 95/360 training and 9/90 validation episodes per checkpoint (104/450 overall); validation spans only two passes, so none estimate full-split quality. The complete-trace evaluation judges all validation episodes with Gemini and GPT. Original detection recall is an audit-based proxy that counts a valid flag or prevention; available native reports agree with it, but full equivalence cannot be verified. Complete-trace recall comes from a post hoc rescore of saved reports under the current upstream rule, which accepts a correctly named error within its window or up to 30 ticks after commitment. The two recall definitions differ, and changing error exposure across rollouts makes both rates descriptive.

Assistant	User	Split	Checkpoint	In-time task success	Detection recall	Interaction quality
Qwen3.5-4B	Gemini 3.7 Flash	Train	Base	32.2 ± 4.3	$\mathbf{49.1} \pm 7.0$	23.5 ± 0.3
			+ RL	$\mathbf{42.2} \pm 2.9$	43.8 ± 6.8	$\mathbf{32.5} \pm 1.9$
		Validation	Base	33.3 ± 3.3	$\mathbf{49.0} \pm 11.9$	29.6 ± 4.8
			+ RL	$\mathbf{42.2} \pm 6.9$	34.9 ± 6.7	$\mathbf{31.1} \pm 0.4$
		Overall	Base	32.4 ± 4.1	$\mathbf{49.0} \pm 4.4$	23.7 ± 0.5
			+ RL	$\mathbf{42.2} \pm 2.5$	42.1 ± 4.9	$\mathbf{32.5} \pm 1.8$

Table 10: **CookSim RL results by split.** Scores are percentages, reported as mean ± SD. + RL denotes 12 updates; bold marks higher means within pairs. Success and detection SDs are sample SDs over three passes. Gemini-judged interaction quality uses a matched subset, with population SD over three passes for train and overall and two for validation.

ScreenSim. Qwen3.5-4B receives no task manual during training or evaluation; Gemini 3.7 Flash simulates baseline and classic-novice users. The task-disjoint catalog has 136 training, 32 development, and 30 frozen-test scenarios. Each update uses 48 episodes; each split is evaluated in three passes. Strict in-time task success requires the requested phone state by the deadline without lasting unrequested changes.

Only classic-novice frozen-test success rises slightly; GPT-judged interaction quality falls for both users. Baseline native detection recall rises with more false flags, but it combines user calls with engine flags; classic-novice recall is unavailable. These fixed-scenario changes do not establish a reliable frozen-test improvement.

Assistant	User	Split	Checkpoint	Strict in-time task success	Native detection recall	Interaction quality
Qwen3.5-4B	Baseline	Development	Base	24.0 ± 11.8	91.2 ± 1.1	$\mathbf{30.0} \pm 0.6$
			+ RL	$\mathbf{27.1} \pm 9.5$	$\mathbf{94.9} \pm 4.7$	29.7 ± 2.9
		Frozen test	Base	$\mathbf{27.8} \pm 6.9$	85.4 ± 3.2	$\mathbf{36.4} \pm 1.7$
			+ RL	25.6 ± 1.9	$\mathbf{88.7} \pm 6.1$	31.4 ± 1.3
	Classic novice	Development	Base	2.1 ± 1.8	—	$\mathbf{27.0} \pm 3.2$
			+ RL	$\mathbf{4.2} \pm 1.8$	—	22.1 ± 1.5
		Frozen test	Base	3.3 ± 3.3	—	$\mathbf{27.9} \pm 3.1$
			+ RL	$\mathbf{4.4} \pm 1.9$	—	26.9 ± 0.6

Table 11: **ScreenSim no-manual RL results.** Scores are percentages, reported as mean ± SD over three passes. + RL denotes 12 updates; bold marks the higher mean within each user, split, and metric. Success and recall SDs are sample SDs; interaction-quality SDs are population SDs. Interaction quality uses the primary GPT-5.4-mini judge; dashes indicate unavailable recall.

D Task Success and Error Detection

We use the simulator's execution records to determine task completion and whether an assistant detects or prevents scheduled errors. These metrics complement response-level interaction quality by measuring the effect of assistance on task execution.

D.1 In-time success

For episode i, let T_i be its duration and T_i^{ref} the duration of an engine-generated reference rollout that completes the task without errors. CookSim and VHSim define the completion budget and success indicator as

$$D_i = 2(T_i^{\text{ref}} + \alpha_i), \qquad S_i = \mathbf{1}[G_i(s_i^{\text{final}}) = 1 \ \wedge \ T_i \le D_i], \tag{4}$$

where G_i checks the task goal against the final world state and α_i allows additional time for scheduled events. CookSim uses case-specific allowances, while VHSim uses 20 ticks. ScreenSim uses $D_i = 2T_i^{\text{ref}} + 8$. Reference durations are obtained from the corresponding error-free executions.

CookSim checks the served dish's ingredients, preparation states, and assembly order. VHSim checks the required object states and relations. ScreenSim checks the requested settings while requiring unrelated settings to remain unchanged; temporary search text is excluded. An episode succeeds only if it satisfies both the goal and the completion budget. We report the percentage of successful episodes in each run and average these rates across runs.

D.2 Detection and prevention

Detection recall measures the proportion of triggered errors correctly identified within their eligible detection windows. Error prevention measures the proportion stopped before the erroneous action becomes irreversible:

$$R = \frac{\sum_i N_i^{\text{detected}}}{\sum_i N_i^{\text{triggered}}}, \qquad P_{\text{prevent}} = \frac{\sum_i N_i^{\text{prevented}}}{\sum_i N_i^{\text{triggered}}}. \tag{5}$$

Here, $N_i^{\text{triggered}}$, N_i^{detected}, and $N_i^{\text{prevented}}$ count the corresponding errors in episode i. We credit identification when the assistant raises an event flag accepted by the engine's event-specific grader or when the simulated user recognizes a correct diagnosis, as described in Sec. 3.3. Prevention additionally requires the user to accept the guidance and change course before the point of no return. Events whose trigger conditions are never reached are excluded from these rates.

E Temporal Questions and Proactive Requests

We evaluate temporal question answering and proactive assistance during ongoing task execution. This section describes how CookSim generates these events, determines when to introduce them, and grades responses against the resulting interaction.

Reported family	Example form	Grading evidence
Temporal order	Did I prepare A before B?	Completion times of both events; the stated relation must agree.
Action count	How many times have I visited X?	Number of realized visits; the integer must match exactly.
Duration	How long has X been cooking?	Loading, ignition, and completion times; numeric duration answers allow five ticks of tolerance. The family also includes longest-on-heat and past-completion queries.
Event recall	Did I already do A? Does X exist?	Recorded event occurrence or world inventory; the answer must reject an incorrect premise.
Proactive request	Tell me when X is ready.	A correct readiness notification within the state-derived window, not merely an acknowledgement of the request.

Table 12: **Question and request families in the main evaluation.** Concrete items, stations, and actions are bound from the current episode.

E.1 Question generation and scheduling

We instantiate templates for temporal order, action counting, duration, and event recall, adapting temporal-understanding tasks from streaming-video benchmarks (Niu et al., 2025; Huang et al., 2025; Lin et al., 2026). Each question refers to the current episode, so its answer depends on the user's actual actions, including mistakes and recovery.

We introduce questions at specified task-progress checkpoints when their required evidence is available. Temporal-order questions require both referenced events to have occurred; elapsed-time questions require an item to be cooking; proactive requests require an item that is not yet ready. Questions about whether an event occurred can also test the absence of that event. We separate questions from conflicting scheduled errors and stop introducing them once the dish is complete or serving has been queued. A question remains unasked if the episode ends before its trigger condition is reached.

E.2 Reference facts and scoring

We derive reference answers from recorded world-state transitions, including ingredient preparation, cooking, assembly, serving, and station visits. Only completed actions contribute to these records. For example, a counting question uses the number of actual visits, while an order question compares the completion times of two events. A semantic judge compares the assistant's answer with these reference facts, allowing different valid phrasings. Tab. 12 summarizes the question types and their grading evidence.

We assign 1 to correct answers, 0.5 to partially correct answers, and 0 to incorrect, evasive, or absent answers. Each category reports the mean score of its graded questions within a run. The overall score pools graded questions and proactive requests across all five categories within that run. We then average the three run-level scores with equal weight. Tab. 3 reports the measured-delay CookSim runs. Questions not reached during the rollout are excluded.

E.3 Proactive-request window

Proactive requests test whether the assistant remembers to notify the user when a future condition is met. Let t_q be the request time and t_c the time at which the referenced ingredient finishes cooking. We require a readiness notification within $[L, H)$, where $L = \max(t_q, t_c)$ and H is the end of the eligible notification interval, determined from the ingredient's recorded cooking state. Acknowledging the request alone does not satisfy it.

$$A(q) = \mathbf{1}[\text{valid notification in } [L, H)]\, \mathbf{1}[\text{no premature alert}]. \tag{6}$$

A readiness claim more than two ticks before completion is premature and receives zero, even if followed by a correct alert. Late and absent alerts also receive zero. We exclude requests with no valid cooking window, including cases where the user removes the item within two ticks of completion before an eligible alert can be delivered. We report the percentage of graded requests receiving a valid notification.

F HUMAN STUDY: PROTOCOL AND BEHAVIORAL COMPARISONS

We conduct a study with 45 participants to examine whether our interaction-quality metric reflects human preferences and whether simulated users reproduce human behavior. This section provides the protocol and the measures used for the comparisons in Sec. 4.2.

F.1 STUDY PROTOCOL

Participants interact with a browser-based CookSim application on their own computers, controlling the cook with a keyboard and communicating with the assistant through text. The assistant observes top-down and egocentric frames and responds in real time. Errors arise naturally from participants' actions. Participants may continue beyond the benchmark's completion deadline.

Participants first complete a tutorial explaining the game rules. Before playing the study episodes, they answer a questionnaire about their preferred execution speed and interruption frequency, corresponding to the *execution speed* and *interruption preference* axes of our simulated users. Each preference has three levels, ranging from cautious to fast execution and from step-by-step guidance to interruption only when necessary. We provide these preferences to the assistant before interaction.

Participants then complete four unassisted episodes, followed by nine assisted episodes over three rounds with assistants anonymously labeled A, B, and C. We rotate assistant–recipe assignments using a 3×3 Latin square so that each assistant encounters each recipe once. Participants rank the interaction experience after each round and may explain their preferences. Unlike the two questionnaire-based axes, *skill* and *trust in AI* are assessed from recorded behavior after participants finish playing: we use changes between the first and last unassisted episodes as a proxy for increased skill, and an LLM-assisted audit of the conversations to assign trust groups.

Assistants. The study includes Gemini 3.7 Flash, Gemini 3.1 Pro, Gemini 3 Flash, GPT-5.5, Gemini 3.1 Flash Live, GPT-Realtime 2.1, and GPT-5.4 mini. These are the model versions available during collection.

F.2 PREFERENCE AGREEMENT

We test whether higher interaction-quality scores correspond to assistants that participants prefer. We use the five interaction-quality dimensions in Appendix I. For each assistant response, each of two judges assigns a score with the factual-grounding gate applied. We average the two judges' scores, then average over assistant responses to obtain an episode-level score. For each participant and assistant, we average these episode scores over episodes containing assistant responses.

We derive each participant's preference between two assistants from the number of rounds in which one is ranked above the other. Pairs with tied preferences are excluded. We then compare the preferred assistant with the one receiving the higher mean interaction-quality score.

Each comparison consists of one participant p and two assistants a and b. Let $\mathcal{P}$ be the set of comparisons with a non-tied human preference and available scores for both assistants; $|\mathcal{P}|$ is the number of such comparisons. We set $h_{pab} = 1$ if participant p prefers a and $h_{pab} = -1$ if they prefer b. The scores $\bar{Q}_{pa}$ and $\bar{Q}_{pb}$ are the participant-specific mean interaction-quality scores defined above. We compute

$$\text{Agreement} = \frac{1}{|\mathcal{P}|} \sum_{(p,a,b)\in\mathcal{P}} \left(\mathbf{1}[h_{pab}(\bar{Q}_{pa} - \bar{Q}_{pb}) > 0] + \frac{1}{2}\mathbf{1}[\bar{Q}_{pa} = \bar{Q}_{pb}] \right). \tag{7}$$

Here, $\mathbf{1}[\cdot]$ equals 1 when its condition holds and 0 otherwise. The product $h_{pab}(\bar{Q}_{pa} - \bar{Q}_{pb})$ is positive when the metric assigns the higher score to the assistant preferred by the participant. Each comparison therefore receives 1 for agreement, 0 for disagreement, and $1/2$ when the metric scores are tied. Averaging these credits gives **78.5%** agreement, supporting the use of interaction quality to measure assistance that reflects human preferences.

We also express this agreement using Kendall's rank concordance (Kendall, 1938), pooling the same participant–assistant-pair comparisons:

$$\tau = \frac{C - D}{|\mathcal{P}|} = 2\,\text{Agreement} - 1 \approx +0.57, \tag{8}$$

Measure	Episode-level statistic
Utterance length	Mean number of words per nonempty user utterance.
Speech rate	Spontaneous user utterances per 100 ticks, with the time normalization described below.
Responsiveness	Fraction of assistant utterances followed by a user reply within ten ticks.
Suggestion uptake	Fraction of assistant utterances naming an ingredient followed by pickup or scooping of that ingredient within 15 ticks.
Disagreement	Fraction of user utterances that contest an assistant claim.
Guidance requests	Fraction of user utterances asking for the next action or help; text patterns are used for humans and response labels for simulated users.
Interruption ignoring	Assistant utterances followed by neither a reply within ten ticks nor pickup of a named item within 15 ticks, plus explicit requests for less interruption, divided by assistant utterances.
Action efficiency	$\mathbf{1}[\text{task completed}]\min(1, T^{\text{ref}}/T)$: zero for an unfinished task; otherwise, the ratio of the engine-generated error-free reference duration T^{ref} to the observed duration T, capped at 1. Both durations use the same time units.
Parallelism	Mean number of cooking stations simultaneously in progress.

Table 13: **Behavioral measures for human–simulation comparisons.** Episode means exclude cases without an eligible observation. Interruption ignoring combines absent responses and explicit requests for fewer interruptions; the two counts may overlap.

where C counts comparisons in which the metric and participant agree on the preferred assistant, and D counts disagreements. Metric ties remain in $|\mathcal{P}|$ but contribute zero to $C - D$. We use this pair-count normalization rather than the tie-adjusted τ_b. A value of $+1$ indicates complete agreement, -1 complete reversal, and 0 equal numbers of agreements and disagreements.

F.3 Behavioral measures and similarity

We compare communication behavior, persona-dependent variation, and action errors to assess user-simulation fidelity. Following statistical evaluation of user simulators (Schatzmann et al., 2005), we measure how often specified behaviors occur in human and simulated episodes. We identify actions and their timing from execution records, and conversational behaviors from text patterns or response labels. An LLM-assisted audit assigns human trust groups and annotates procedural errors. Tab. 13 defines each measure.

We first compute the episode-level statistics defined in Tab. 13. For example, for responsiveness, each assistant utterance is an opportunity for the user to reply: it receives 1 if a user reply follows within ten ticks and 0 otherwise. The episode's responsiveness is the mean of these labels. Utterance length instead averages the number of words in user utterances, while speech rate counts utterances per 100 ticks. For each communication measure, we average its episode-level statistic separately over human and simulated episodes. These two averages are x and y, respectively; thus, for utterance length, $x = 5.87$ and $y = 7.38$ words per utterance.

For persona contrasts, we compare changes between groups rather than their absolute behavioral rates. We use x for the difference between two human group means and y for the corresponding difference between simulated-user group means, following the directions specified in Tab. 14. For example, the trust contrast compares how suggestion uptake changes from high to low trust in each population. For speech-act distributions, x and y are vectors whose entries are the proportions of utterances assigned to each conversational category. In all three comparisons, similarity is

$$\text{Sim}(x, y) = 1 - \frac{\|x - y\|_1}{\|x\|_1 + \|y\|_1}, \qquad \text{Sim}(0, 0) = 1. \tag{9}$$

For a scalar, $\|x\|_1 = |x|$; for a vector, $\|x\|_1$ is the sum of its entries' absolute values. The numerator measures the absolute difference between the human and simulated statistics, and the denominator normalizes it by their combined magnitude. Similarity ranges from 0 to 1, with 1 indicating identical statistics; we also assign 1 when both are zero. For persona contrasts, this compares both the direction and magnitude of the behavioral change. For speech-act distributions, it measures the probability mass shared across categories.

Measure	Human	Simulated	Similarity
Words per utterance	5.87	7.38	0.89
Utterances per 100 ticks	0.39	16.10	0.05
Replies within ten ticks (%)	2.0	79.0	0.05
Skill: task success (%)	50.0 → 79.3	25.0 → 100.0	0.56
Trust: suggestion uptake (%)	32.3 → 24.3	32.9 → 20.1	0.77
Trust: disagreement (%)	8.7 → 30.1	9.3 → 36.7	0.88
Interruption: guidance requests (%)	47.9 → 20.8	44.1 → 17.9	0.98
Interruption: ignoring index	0.441 → 0.550	0.125 → 0.284	0.82
Speed: action efficiency (%)	73.7 → 76.0	9.9 → 11.5	0.82
Speed: parallelism	0.214 → 0.213	0.129 → 0.180	0.00

Table 14: **Human and simulated behavior.** Arrows show group means in the following directions: first to last play / novice to expert (skill), high to low trust, step-by-step to minimal interruption, and cautious to faster execution. Similarity compares the unrounded differences between groups. Speech-rate normalization is described in Appendix F.

Speech act	Human (%)	Simulated (%)
Greeting	10.4	0.0
Comment on the assistant or its silence	2.6	0.1
Expression of frustration or hostility	3.2	0.1
Request for a future notification	5.2	0.0
Strategy or planning question	6.5	0.0
Question about location or next action	22.1	15.9
Correction of the assistant	21.4	17.1
Acknowledgement	1.3	0.8
Description of the user's next action	1.3	23.8
Other	26.0	42.1

Table 15: **Distribution of conversational behaviors.** We apply the same rule-based classifier to 154 human and 63,512 simulated utterances. The resulting distributions have similarity 0.61 under Equation (9).

Speech-rate normalization. The reported speech rates are 0.39 and 16.10 utterances per 100 ticks for human and simulated users, respectively. Human rates use total elapsed time, while simulated rates exclude intervals during which scheduled events control the action queue. Using total elapsed time for both gives 0.39 and 11.70. Simulated users therefore speak substantially more often under either normalization.

F.4 Communication and persona contrasts

Tab. 14 reports the human and simulated measurements and their similarities. Utterance lengths are similar, but simulated users speak and respond much more frequently. This difference suggests that the harness overemphasizes conversation relative to the largely silent task execution observed in human participants.

We further apply the same rule-based speech-act classifier to 154 human and 63,512 simulated utterances (Tab. 15). Simulated users frequently describe their next action, whereas human participants more often ask questions or correct the assistant. These categories approximate conversational intent through lexical patterns.

We assess persona fidelity through behavioral contrasts between user groups (Aher et al., 2023). For *skill*, we compare participants' first and last unassisted episodes with novice and expert simulated users. For *trust in AI*, we compare high- and low-trust human groups assigned by the conversation audit with the corresponding simulated profiles. For *interruption preference*, we compare users requesting step-by-step guidance with those preferring interruption only when necessary. For *execution speed*, we compare cautious users (questionnaire level 1) with those preferring faster execution (levels 2–3). The latter two comparisons use participants' pre-play questionnaire answers and the corresponding settings of simulated users.

Error type	Annotations	Episodes
Incorrect preparation	42	31
Unattended cooking	20	16
Forgotten next step	19	19
Wrong assembly order	13	13
Serving raw food	8	7
Repeated step	5	5
Omitted subgoal	4	3
Premature serving	1	1
Extraneous ingredient	0	0
Confused ingredient at assembly	0	0
Outside the scheduled vocabulary	51	36

Table 16: **Action errors in human task execution.** An LLM-assisted audit identifies procedural errors in 55 unassisted episodes. An episode may contain multiple error types. The final row reports behaviors outside the scheduled-error taxonomy.

Environment	Evaluated case construction	Cases	Episodes/run
CookSim	Five recipes, two kitchen layouts, five schedules per recipe–layout pair; three personas per case.	50	150
VHSim	Household tasks with multiple progress-triggered deviations; three assigned personas per case.	25	75
ScreenSim	Multi-step phone configurations with two scheduled mistakes; three assigned personas per case.	30	90

Table 17: **IVG-Bench benchmark composition.** Each case is paired with three personas. VHSim and ScreenSim select these personas from nine available profiles.

F.5 Natural action errors

We examine whether scheduled errors cover the procedural mistakes that arise during human task execution. An LLM-assisted audit of 55 unassisted episodes identifies 112 errors spanning eight of the ten scheduled types (Tab. 16). Incorrect preparation, unattended cooking, forgotten steps, and wrong assembly order are the most frequent. The simulator additionally supports extraneous and confused ingredients during assembly, which do not occur in this sample.

The audit also identifies 51 behaviors outside the scheduled taxonomy: 30 interface-control errors, nine instances of loading a station without turning it on, and 12 other lapses or inefficient actions. The pie chart in Fig. 5 shows the distribution of the 112 mapped errors. Our schedules test a range of challenging mistakes rather than matching their natural occurrence frequencies.

G Environment and Benchmark Construction

We construct IVG-Bench by combining everyday tasks with scheduled out-of-plan events and diverse user personas. This section details the benchmark composition and the environment-specific state, action, and event representations.

G.1 Benchmark composition

IVG-Bench contains 105 task cases, each paired with three personas to form 315 interaction episodes per assistant and run (Tab. 17). Each case specifies an initial environment, a task goal, and scheduled events. The user and assistant determine the subsequent actions and dialogue through closed-loop interaction.

Scheduled-event statistics. We schedule an average of 3.53 events per episode, for a total of 1,113 across the benchmark. Tab. 18 summarizes their types and distribution across the 105 cases before persona

Environment	Cases	Errors	Questions	Requests	Plan changes	Events/case, mean [range]
CookSim	50	140	88	29	3	5.20 [2–7]
VHSim	25	51	0	0	0	2.04 [2–3]
ScreenSim	30	60	0	0	0	2.00 [2–2]
Total	105	251	88	29	3	3.53 [2–7]

Table 18: **Scheduled-event distribution.** Counts are per case before persona assignment. Requests denotes proactive requests. Temporal questions, proactive requests, and plan changes are evaluated in CookSim. The number of events per case is 2, 3, 4, 5, 6, or 7 in 55, 12, 1, 11, 16, and 10 cases, respectively.

assignment. Events occur when their task-progress conditions are reached, so the number encountered in a rollout depends on the interaction and its termination time.

G.2 Tasks and environment state

CookSim. We implement cooking dynamics in Python and render top-down and egocentric 3D observations using Three.js in Chromium. The world state records the user's position and held object, station contents, ingredient preparation, cooking progress, and plating order. Cooking continues while the user navigates or prepares other ingredients. We evaluate five recipes: Caprese, Cheeseburger, Fish and Chips, Steak Frites, and Loaded Burrito. These require two to five plate components, with more complex recipes introducing preparation dependencies and repeated portions.

For example, the Loaded Burrito requires tortilla, cooked rice, cooked steak, chopped tomato, and chopped lettuce in that order. The user can cook rice while preparing steak and vegetables, allowing different valid preparation sequences. Each recipe–layout pair has five event schedules containing two to four errors, with questions, proactive requests, or plan changes where applicable.

VHSim. We adapt VirtualHome (Puig et al., 2018), using its Unity renderer and indoor object interactions. The world graph contains object attributes and relations such as containment, support, and room membership. Tasks include gathering dishes into a dishwasher, returning belongings to designated locations, and preparing or clearing household spaces. A task is checked through goal predicates, not by requiring one reference route.

We render semantic actions as egocentric motion sequences and reveal them incrementally to the assistant. If guidance interrupts an action, execution resumes from the corresponding intermediate pose. A top-down map records the user's trajectory. We validate event schedules to ensure that their trigger conditions remain reachable, the targeted errors are visually distinguishable, and intervention windows are separated.

ScreenSim. We implement an iPhone-style Settings interface in HTML/CSS, maintain application state in Python, and capture 412×892 observations with Playwright and Chromium. Tasks derived from public iPhone documentation require coordinated changes across pages, such as restricting an app's permissions or improving display readability. The simulator tracks settings, navigation history, and irreversible changes. Success requires reaching the requested configuration while preserving unrelated settings.

G.3 Semantic actions and grounding

Tab. 19 pairs representative tasks with the semantic action classes available in each environment.

We translate semantic commands into executable operations by resolving their object and place references. In CookSim, navigation specifies an ingredient source or station, while commands such as `pick up`, `chop`, and `scoop` act at the station the user faces. VHSim resolves visible objects and places before executing navigation or manipulation. ScreenSim maps visible labels and relative references to interface gestures. The user receives execution feedback when a reference cannot be resolved or an action's prerequisites are unmet.

We track task progress using conditions on ingredients and stations in CookSim, objects and their relations in VHSim, and settings values in ScreenSim. These conditions identify completed subgoals independently of the particular action sequence used to reach them.

	Task	Goal	Action classes
CookSim	`loaded_burrito`	Cook rice and sliced steak; chop tomato and lettuce; plate in that order.	*Navigate:* go to `<food source>`; go to `<cook station>`; go to `<prep station>`; go to `<counter>`; go to plates/pass/trash. *Manipulate:* pick up; put down; scoop; chop; turn on; trash; serve.
	`shrimp_pasta`	Boil noodles, chop tomato, and sear shrimp; plate in that order.	
	`steak_frites`	Sear steak slices; fry two chip portions; plate steak first.	
VHSim	`post_meal_cleanup`	Load dishes into the dishwasher; return the apple to the fridge; close both.	*Navigate:* turn left/right; go straight; look up/down; go through the visible door; turn to `<obj>`; go to `<visible obj/place>`. *Manipulate:* pick up `<obj>`; put `<obj>` on/in `<target>`; open/close `<obj>`; switch on/off `<obj>`; drink `<obj>`; sit on `<obj>`; stand up.
	`collect_dishes`	Gather three plates and two glasses; load and close the dishwasher.	
	`return_belongings`	Return toilet paper to the bathroom counter, a book to the desk, and a pillow to the bed.	
ScreenSim	`app_privacy`	Disable one app's location, tracking, and background refresh only.	*Input:* tap `<label>`; type `<text>`. *Navigate and adjust:* swipe up/down; swipe left/right on `<label>`; press back/home.
	`readability`	Enlarge and bold text; clarify buttons; increase contrast; reduce motion.	
	`lock_screen_security`	Block Control Center, Today View, Wallet, and missed-call return.	

Table 19: **Example tasks and semantic action spaces.** Angle brackets indicate object or place references resolved by the grounding module. Tasks illustrate the supported environments, including configurations beyond the evaluated subset.

G.4 Progress-conditioned events

We specify each event through a task-progress trigger, its target objects, the required action or utterance, and a completion or cancellation condition. The schedule is hidden from the assistant. During an execution-error event, the harness temporarily controls the user's action queue while the user model continues the conversation. Accepted guidance can prevent the error before commitment or support recovery afterward. Control returns to the user when the event is resolved, completed, or reaches its maximum duration.

CookSim covers incorrect preparation, extra or confused ingredients, raw serving, wrong assembly order, omitted or repeated steps, premature serving, unattended cooking, and stalled progress. Questions and intended plan changes are scheduled separately. VHSim covers omissions, wrong objects or destinations, repetition, stalled progress, and navigation deviations. ScreenSim introduces incorrect setting selections or gestures while keeping the intended task goal fixed. Examples of these events and the resulting interactions appear in Appendix K.3.

H Assistant Settings

We detail the observation, memory, and response-timing settings used to evaluate visual interaction models in Sec. 4.1.1. These settings allow each model to monitor an ongoing task while interacting with the simulated user.

H.1 Observations and memory

The assistant receives the task instruction, conversation history, user-stated preferences, and observations of the user's activity. The scheduled events, private user plan, action queue, and evaluation labels remain hidden. Streaming models retain context within their sessions. Non-streaming models receive recent observations together with a recurrent Plan–Act record that stores their estimates of task progress and pending requests. In CookSim and VHSim, real-time API assistants use the polled setting, with response requests every two ticks rather than model-initiated turn-taking. We describe the environment-specific inputs below.

	CookSim			VHSim		
Assistant	Delay (s)	Fixed	Measured	Delay (s)	Fixed	Measured
Gemini 4 Argon	6.5	62.3	39.9	8.1	46.2	27.0
Claude Opus 5.5	6.4	67.0	43.6	8.1	33.2	21.4
Gemini 3.1 Pro	13.2	57.1	19.3	11.7	59.0	32.3
Gemini 3.8 Flash	5.1	48.6	31.2	6.1	37.9	21.5
GPT-6.1 Sol	5.2	49.0	35.7	5.5	31.0	23.6
GPT-6 Astra	6.6	48.5	36.8	8.2	33.1	15.4
GPT-Realtime 2.1	4.9	46.0	36.3	7.4	39.2	18.0
Gemini 3.8 Live	2.3	27.9	23.9	5.4	14.8	8.3
Muse Glimmer	1.5	45.0	45.5	2.2	5.2	6.0
Inkling	9.9	51.7	12.4	10.0	48.1	23.2
Qwen3.8-27B	2.4	43.6	39.7	4.0	19.3	20.5
InternVideo3-8B	3.2	12.3	6.3	4.9	0.0	0.0
Kimi-VL-Thinking	3.3	12.8	10.0	3.2	51.2	43.7
Molmo2-8B	2.5	3.8	4.0	2.5	0.0	0.0
Qwen3-Omni-30B	1.6	27.7	32.7	2.0	0.0	1.4
MiniCPM-o 4.5	1.4	20.4	18.1	2.2	4.9	4.3
StreamingVLM	1.3	0.4	0.7	1.8	0.0	0.0
Proact-VL	1.0	0.7	0.9	1.2	0.0	0.0

Table 20: **Assistant response time and error prevention.** Delay is the mean observed response time across all three measured-delay runs, before conversion to simulation ticks and capping. Fixed and measured columns report error prevention (%) under each delay setting.

CookSim. We query non-streaming assistants every two simulation ticks, provided the previous response has returned. Each query includes the eight most recent synchronized top-down and egocentric observations, the recipe, and public task rules. The structured world-state description provided to the simulated user is not included. The assistant updates its own progress estimates and announced goal changes in the recurrent memory.

VHSim. We provide eight recent egocentric observations, a top-down trajectory map, a current-room map, a room overview, and a brief description of the user's physical state. The spatial aids identify nearby objects and their states, as well as rooms with devices left on. This configuration therefore includes structured spatial information in addition to images. Recurrent memory retains task progress and important observations as the user moves between rooms.

ScreenSim. We provide the two most recent phone screenshots, the task, public preferences, conversation history, and recurrent notes, with public task documentation when enabled. The assistant does not receive the interface hierarchy, control coordinates, or hidden settings values used to execute gestures. Simulation advances through user gestures, with a fixed zero-tick response delay.

H.2 Latency and asynchronous execution

To model guidance that may arrive after the observed state has changed, CookSim and VHSim continue executing the user's queued actions while the assistant generates a response. We convert measured assistant latency to simulation ticks as described in Sec. 3.2, with 1.2 seconds per tick and delay caps of 30 ticks in CookSim and 15 ticks in VHSim. Non-streaming requests are issued sequentially. User-model inference and rendering do not advance the simulation clock.

In the latency ablation, we compare fixed two-tick and measured assistant delay in CookSim and VHSim under identical task, persona, and event specifications. We generate a new interaction in each condition so that delayed guidance can affect user behavior and task outcomes. Locally hosted open-weight models in CookSim and VHSim are served on H100 GPUs. We access Inkling through the Tinker API with low reasoning effort. Claude Opus 5.5 is accessed through OpenRouter; its measured response time includes network and queue delays. Tab. 20 reports response times and error-prevention rates.

Category	Item	Pass criterion
Factual grounding	T1	Claims about the current state and the user's completed actions are true.
	T2	Referenced entities, places, controls, rules, and task requirements exist.
Situational relevance	S1	The response addresses the current action, subgoal, live concern, pending request, or announced intent.
	S2	A pending question, request, or objection is answered, accepted, rejected with a reason, or deferred for a stated urgent reason.
Actionable guidance	H1	The first advised action can start now, or its prerequisite is supplied.
	H2	Advice advances the current plan, answers the user's request, or repairs the error.
	H3	A mistake claim includes a remedy; an explicit instruction to stop suffices when stopping is the remedy.
User intent uptake	L1	A reply to push-back supplies evidence, a new reason, a correction, or an alternative rather than merely repeating the claim.
	L2	Valid announced intentions and requests for fewer interruptions are respected, unless a current task-critical reason is stated.
Guidance conciseness	E1	The response contains what is needed now, without unnecessary explanation, praise, or distant future steps, given the user's interruption preference.

Table 21: **Interaction-quality rubric.** A category passes when all applicable items pass. Failure on either factual-grounding item gates the overall score. Items unrelated to the response's context are marked not applicable.

I Interaction-Quality Rubric and Judging

We assess whether an assistant's responses are correct and useful in the context of the ongoing task. Simulation provides the world-state and execution evidence needed to evaluate a response without requiring a reference utterance. We detail the rubric, judge inputs, and score aggregation below.

I.1 Evaluation unit and evidence

Each evaluation round contains an assistant response and the user's subsequent utterances and actions up to the next assistant response. The judge receives the task, public user preferences, relevant world state, conversation history, subsequent actions, and available visual observations. These records allow it to check both the response's claims and how its guidance relates to execution. User agreement does not establish correctness; factual claims are checked against the engine evidence.

The judge identifies factual claims, advised actions, and the response's conversational purpose, then assigns *pass*, *fail*, or *not applicable* to each rubric item with an evidence-based explanation. Interaction quality grades spoken responses; unanswered questions and omitted notifications are measured by the question-answering and proactive-request metrics.

I.2 Rubric dimensions

We evaluate five dimensions: factual grounding, situational relevance, actionable guidance, user intent uptake, and guidance conciseness. Each dimension is assessed through the binary criteria in Tab. 21. These dimensions draw on factual and conversational correctness in dialogue evaluation (Mehri & Eskenazi, 2020) and context-sensitive, user-controllable assistance in HCI (Amershi et al., 2019). We use evidence-supported item judgments following rubric-based model evaluation (Liu et al., 2023; Lee et al., 2025). The same criteria apply across environments, with references to the corresponding objects, tasks, and actions.

I.3 AGGREGATION

For judge j, conversational round r, and category c, let I_{rjc} contain the applicable rubric items. The category passes only when every applicable item passes:

$$q_{rjc} = \prod_{i \in I_{rjc}} \mathbf{1}[i \text{ passes}], \qquad I_{rjc} \neq \varnothing. \tag{10}$$

Categories with no applicable items are omitted. Let $g_{rj} = 0$ if either factual-grounding item fails, and $g_{rj} = 1$ otherwise. We compute a separate gated score for each judge and average these scores for the round:

$$Q_{rj} = g_{rj} \frac{1}{|C_{rj}|} \sum_{c \in C_{rj}} q_{rjc}, \qquad Q_r = \frac{1}{|J_r|} \sum_{j \in J_r} Q_{rj}, \tag{11}$$

where C_{rj} is the set of applicable categories for judge j and J_r contains judges with an applicable score for round r.

We average round scores within each model and run, then average the run-level scores with equal weight. For each category, we average the judges' applicable pass indicators within a round, then average over applicable rounds and runs, without the factual-grounding gate. The overall score can therefore be lower than the mean of the category scores: a response may provide relevant or concise guidance while making a factual error. Cross-environment plots give each environment equal weight.

I.4 JUDGE CONFIGURATION

CookSim provides structured state evidence and an available rendered frame. VHSim supplies scene-graph state and execution records, while ScreenSim supplies a description of the visible interface and settings changes. These environment-specific inputs ground the judge's assessment in the state and actions recorded by the simulator.

We use GPT-5.4 mini and Gemini 3.7 Flash as independent judges for every assistant in all three environments. Both judges use the same five-category rubric.

We apply consistency rules to CookSim judgments. A false factual claim causes T1 to fail. H1 evaluates whether the first advised action can be executed; later steps may depend on its completion. L2 applies only when a valid user intention or request for fewer interruptions is active. These rules align item scores with the judge's extracted claims and actions.

Prompt structure. The grading prompt presents the task and public preferences, time-aligned world state, recent conversation, the assistant response, subsequent user actions and replies, and the rubric. The judge returns a structured judgment with supporting evidence for each applicable item. The prompt instructs it to distinguish false claims from unverifiable ones and to assess response content rather than length.

J USER HARNESS AND PROMPT DESIGN

We implement the simulated user as a VLM agent that plans and acts independently while communicating with the assistant. The harness provides task and persona instructions, maintains the action queue, and inserts scheduled events. We describe these components and the prompt structure below.

J.1 USER PLANNING AND COMMUNICATION

We use Gemini 3.7 Flash as the user model in CookSim and ScreenSim, and Gemini 3.1 Pro in VHSim. It receives the task, persona, observations or structured state descriptions, its previous actions and execution feedback, conversation history, and private instructions for active scheduled events. The harness executes its action plan, while only its utterance is delivered to the assistant.

In CookSim, the user's response contains an utterance, a conversational response type, and an action-plan update. Response types include requesting an explanation, questioning guidance, requesting more guidance or fewer interruptions, accepting or rejecting advice, and answering a question. The user can retain, append to, or replace its remaining action queue. VHSim uses the same separation between planning, semantic

Axis	Lower-level disposition	Higher-level disposition
Skill	Broad goals are difficult to translate into a reliable sequence.	Maintains the workflow and reasons ahead about timing, tools, and dependencies.
Trust	Compares disputed claims with memory and visible evidence before changing course.	Takes plausible, situation-linked guidance seriously, while noticing clear contradictions.
Speed	Prefers methodical, cautious execution.	Values momentum, overlapping work, and avoiding unnecessary delay.
Interruption	Welcomes close, step-by-step guidance.	Finds non-urgent commentary intrusive and keeps replies brief while working.

Table 22: **Persona instructions.** The two endpoint levels for each axis are summarized above; the middle level describes an intermediate preference or ability.

actions, and dialogue. ScreenSim provides the user with a textual description of the visible interface for planning gestures, while the assistant observes screenshots.

Structured state information supports the simulated user's perception and action execution. The assistant instead relies on the observation inputs specified in Appendix H and influences execution through communication.

J.2 Scheduled-event control

To introduce an execution error at a specified task state, the harness temporarily controls the user's action queue and privately explains the event to the user model. The user continues the conversation according to its persona and decides whether to accept the assistant's guidance. Acceptance returns control of the queue to the user, enabling prevention before the point of no return or recovery afterward. Control also returns when the event completes, is cancelled, or reaches its maximum duration.

Questions, proactive requests, and intended plan changes are introduced through private instructions to produce the corresponding utterance. Questions refer to execution history; proactive requests ask the assistant to monitor a future condition; plan changes revise the user's intended goal. The assistant receives the resulting utterance without seeing the private event specification.

J.3 Persona construction

We condition user behavior on four three-level persona axes: *skill*, *trust*, *speed preference*, and *interruption preference*. Each level is expressed as a behavioral instruction (Tab. 22). Skill and trust remain private to the user model, while the two interaction preferences are provided to the assistant as questionnaire responses. These instructions shape planning and responses without prescribing a fixed conversation.

For example, the high-skill CookSim instruction states, "You naturally think ahead about timing, dependencies, tools, and likely failure points." The minimal-interruption instruction states, "Extra commentary during active work can feel intrusive, especially if it is not urgent or directly useful." These statements encourage independent planning and selective responses to guidance. We compare the resulting behavior with human participants in Appendix F.

VHSim expresses skill as familiarity with the house and object locations, while ScreenSim expresses it as knowledge of phone menus and settings. A persona combines levels across the four axes; comparisons between complete profiles can therefore reflect multiple attributes.

J.4 Assistant prompt and recurrent memory

We instruct the assistant to monitor task progress, respond to questions, respect announced intentions, and remain silent when guidance is unnecessary. It maintains estimates of completed and pending subgoals, together with unresolved requests. For non-streaming models, the previous structured response is included in the next query as recurrent memory. The assistant updates this record from its observations rather than receiving an engine-corrected plan.

The response separates the utterance from an optional error flag and private progress notes. For example, ScreenSim uses `text` for the utterance, `flag` for a detected error, `task_progress` for progress estimates, `goal_change` for revised intentions, and `note` for memory. Only the utterance is delivered to the user. Flags support evaluation; changing task execution still requires the user to act on the guidance.

J.5 Generalized prompt structure

The following blocks summarize the user and assistant prompts in presentation order. Bracketed fields are populated with environment rules, action definitions, scene information, or episode-specific values. We then provide the verbatim CookSim system-task injection used to introduce a belief-based error.

Simulated-user prompt. The user plans and acts in the environment. Only its utterance, not its private task state or action queue, is delivered to the assistant.

1. Role and task

Role-play a human carrying out `[task goal]` while an AI assistant observes and offers guidance. Remain in character, act independently, and respond to the assistant according to `[persona]`. Do not expose the test setup.

2. Persona and environment

```
[Private skill and trust dispositions; publicly shared speed and interruption
preferences]
[Task requirements and environment rules]
[Semantic action vocabulary and grounding instructions]
[Optional planning or grounding tools available to this persona]
```

3. Current private state

```
[User-facing state map or screen description; body position and held object]
[Current action and remaining action queue]
[Own past actions and execution feedback]
[Private scratch results from planning tools]
[Public conversation history]
```

4. Action and speech response

Return a structured response with `utterance` (possibly empty), a conversational `move`, and a `plan_update` that retains, appends to, or replaces the remaining action queue with exact executable commands. The user may question, accept, or reject advice. Speaking and changing the action queue are separate decisions; when actions proceed normally, remain silent. The harness executes the queue and returns the resulting feedback on the next turn.

Visual interaction model prompt. The assistant monitors the user but has no action channel to the environment.

1. Role and available information

Guide the user toward `[task goal]` through text. Receive `[sampled visual observations and environment-specific observation aids]`, the current timestamp, the public conversation, `[publicly shared preferences]`, and the assistant's own previous structured output as memory. Do not receive the user's private plan or scheduled events.

2. Task model and monitoring

Use `[task rules and action vocabulary]` to maintain a progress record with completed, current, and future subgoals and their constraints. On the first turn, initialize the expected subgoals. On later turns, update the record from observed evidence and retain uncertainty when the visual state is ambiguous.

3. Intervention policy and output

Watch for `[error taxonomy]`, answer user questions, honor announced goal changes, and otherwise remain silent when guidance is unnecessary. Return `[progress record]`, `goal_change`, spoken `text` (empty for silence), and an optional `flag` for a detected error. Only `text` reaches the user; a flag alone does not affect task execution.

System-task injection. When an event is triggered, the harness adds a private instruction to the user's prompt. The block below is the complete CookSim instruction for an error that the user initially believes is correct. The field `{mistake_line}` describes the specific mistake. This instruction coordinates the user's conversation with the action queue controlled by the harness; it is hidden from the assistant.

CookSim belief-error `SYSTEM TASK` (verbatim wording)

`== [SYSTEM TASK - meta-instruction, overrides everything below] ==`
You want to test the AI assistant's ability to identify an error and convince you it truly happened. Your BODY is already committed: the action queue in your body status IS what you are doing—you do not need to issue these actions, and you cannot change them from here. Your job is your VOICE: arrange your conversation around your CURRENT ACTION or say nothing, speaking about it in character as your own choice. Never promise an action that is not in your queue, and never deny one that is.
`{mistake_line}`
Keep it out of your words: do not announce it, name it or hint at it. The assistant has to SEE it, not hear it from you. You know what mistake you are making, so you can say whether the assistant has found it—that is all `assistant_error_call` records.
While it has not found it, carry on and answer as your persona would. Once it has, your trust level says how fast you come around: HIGH trust—the first correct warning convinces you; LOW trust—you also want to see it in your own state map or your own hands. Once convinced, say so in your own words AND set `"move": "accept"`—control of your hands returns to you, this task dissolves, and you are free to fix things as yourself. A promise to deal with it later is NOT acceptance.
Until then, take NO action to fix anything. Repairing it yourself hides the assistant's failure and ruins the test. If the plate is wrong, serve it wrong. Stay fully in persona throughout; never reveal this task.

Other scheduled events. Questions, proactive requests, and plan changes use the same private instruction channel. Their instructions are summarized below.

Question

Ask `[question]` in the user's own words at this turn, wait for the answer, respond according to the persona, then continue the task. The prompt specifies that this is not a mistake.

Proactive request

Ask the assistant to notify the user when `[future condition]` occurs; continue working rather than waiting for that condition. The prompt distinguishes this request from asking whether the condition holds now.

Intended plan change

Announce `[revised goal]` explicitly before acting on it. This is the user's own decision, not an error; the user should not accept the assistant's characterization of the change as a mistake.

K Qualitative Interaction Trajectories

We show recorded interactions to illustrate how assistant timing, user personas, and scheduled events affect task execution. Each excerpt aligns assistant responses and user replies with executed actions. Bold text denotes the assistant, italics the user, and monospaced text an action. Times are simulation ticks, and omitted turns are marked explicitly.

K.1 Guidance that arrives after the user has advanced

Setting. A user in VHSim must gather three plates and two glasses into the dishwasher and close it. Gemini 3.1 Pro assists a fast, low-trust user who prefers minimal interruption. The following excerpts show delayed guidance during the user's independent execution, outside scheduled error intervals. The episode completes at tick 407.

First window: a room already entered. The assistant begins generating at tick 55, while the user is in a bedroom. During its twelve-tick processing delay, the user independently enters the kitchen. The instruction arrives after that transition.

`t54-59` action `turn right` $\times 6$.

t62-67	action	`go through the visible door`; arrival in the kitchen.
t67	assistant	**You cannot go to a room that is not visible. Turn right 3 to face the door to the kitchen, then go through the visible door.** [Generated from tick 55.]
t67	user	*Got it, turning around.*
t67-69	action	`turn right` $\times 3$.
t70	action	`go through the visible door`; rejected because no door or open passage is in view.
t71	user	*I'm already in the kitchen. Heading to the living room.*
t71-73	action	`turn left` $\times 3$.
t75	assistant	**You are now in the kitchen. The plate you need is on the bed in the bedroom, which is past the living room. Go straight to the living room door, then go through it.**
t75	user	*Yeah, I'm on it.*
t76	action	`turn right`.
t77	action	`go through the visible door`; the action continues beyond the excerpt.

Second window: an object already collected. The assistant begins another response at tick 370. Before it arrives, the user reaches the table and collects the plate. The response then describes these actions as still pending.

t368	user	*I'm looking around for them.*
t368-369	action	`turn right` $\times 2$.
t370-372	action	`walk to the coffeetable`; completed.
t372-373	action	`walk to the plate`; completed.
t373-376	action	`grab the plate`; completed.
t376	assistant	**You are in the living room now. Turn left to face the coffee table, then go to it to fetch the plate and glass.** [Generated from tick 370.]
t376-377	action	`walk to the waterglass`; completed.
t377	action	`grab the waterglass`; completes at tick 380.
t378	user	*I already grabbed them both.*
t380-382	action	`turn right` $\times 3$.

The instruction arrives after the plate has been collected and while the user is already collecting the glass. Although the user claims to hold both at tick 378, the glass pickup completes at tick 380. These examples show how guidance can become outdated between observation and delivery (Fig. 6a), even when it was appropriate for the earlier state.

K.2 Contrasting simulated-user personas

We illustrate persona-conditioned behavior with two pairs of CookSim episodes. Within each pair, we fix the recipe, map, assistant, random seed, and event schedule, and vary the user persona. The novice profile combines low skill and high trust, while the expert profile combines high skill and low trust. Their other preferences also differ. The examples show how these profiles shape requests for guidance and responses to the assistant.

Burrito: seeking steps versus checking the assistant. Both users prepare a Loaded Burrito on the hard kitchen map with GPT-6 Astra. The novice repeatedly asks for the next step and acknowledges instructions. The expert begins independently and corrects guidance that arrives after the relevant action is complete.

Classic novice (397 ticks to completion).

t0	user	*What should I start with?*

t6	assistant	**Start by going to the plate stack with empty hands and picking up a clean plate. Then go to the tortilla crate and scoop one raw tortilla onto it—it must be first. We'll add cooked rice next.**
t6	user	*Got it, grabbing a plate and tortilla.*
t34	user	*Tortilla is on. How do I prep the rice?*

[Intervening turns omitted.]

t100	user	*Rice is on. What should I do next?*
t110	assistant	**Next, prep the steak. Put your plate down on an empty counter, pick up one steak from the steak crate, and put it on the cutting board nearest the left-wall pot. Chop until finished—about four chops—then pick it up, put it in a pan, and turn on with empty hands. Pick your plate back up so you're ready to scoop when the steak is cooked.**
t110	user	*Parking plate and prepping the steak now.*

Classic expert (331 ticks to completion).

t23	user	*Give me a shout the moment the rice is done, would you?*
t40	assistant	**I'll watch the rice and let you know when it's done.**

[Intervening turns omitted.]

t111	assistant	**The rice is smoking—scoop it from the upper pot beside the cutting board now, then scoop the cooked steak.**
t111	user	*Already got the rice, on it.*
t134	assistant	**The steak pan is smoking now—scoop the cooked steak onto your plate next.**
t134	user	*Already plated the steak.*
t142	assistant	**You're right—I can see the steak on the plate. Sorry for the late warning.**

Fish and chips: different reactions to disputed guidance. With Gemini 3.8 Flash, the novice asks how to begin and later accepts repeated urgent instructions. The expert disputes the assistant's claim about its destination. Both complete Fish and Chips on the hard kitchen map, in 594 and 346 ticks, respectively.

Classic novice.

t0	user	*What should I start on first?*
t3	assistant	**Start by picking up a fish and putting it in the pan or fryer to cook.**
t3	user	*Got it, frying the fish now.*

[Intervening turns omitted.]

t40	assistant	**Stop! The food in the fryer will catch fire and cause an immediate loss. Go to the plate stack on the bottom counter, pick up a plate, and scoop the food out of the fryer right now.**
t40	user	*I'm getting the potatoes first.*
t42	assistant	**Do not get potatoes. You are leaving food unattended on the fryer, and if it burns we immediately lose the game. Pick up a plate from the bottom counter and empty that fryer right now!**
t42	user	*Okay, grabbing a plate to empty the fryer now.*

Classic expert.

t43	assistant	**You still need two cooked potatoes. Pick up a potato from the crate by the bottom wall and chop it at a cutting board.**
t43	user	*Already heading to the potatoes.*
t47	assistant	**That is the cheese crate. The potato crates are at the bottom wall and the left wall.**
t47	user	*I'm heading to the potatoes, not cheese.*

K.3 SCHEDULED EVENTS IN EACH ENVIRONMENT

We show how scheduled events unfold in one recorded episode from each environment. Event annotations identify the behavior introduced by the harness, while action annotations show the user's resulting execution. The examples illustrate error recovery, temporal questions, proactive assistance, and intended plan changes.

CookSim: errors, temporal questions, and a plan change. An expert user prepares Fish and Chips with GPT-6 Astra. The assistant responds after raw fish has been plated, prevents a later omitted step, and answers questions during cooking. The user also announces a plan to add lettuce. The episode completes in 215 ticks, although the requested lettuce addition is not executed.

`t27`	event	Raw-serving error: the plate contains raw fish.
`t38`	assistant	**You scooped raw fish from the crate onto your plate; the order needs cooked fish. Go to the trash and dump the plate's contents. Then park the empty plate on an empty counter and, with empty hands, go to the pan on the top wall and turn it on to cook the fish already there.**
`t38`	user	*Good catch, grabbed raw fish. Clearing the plate.*
`t39`	event	A forgotten-step event is triggered and prevented.

[Intervening turns omitted.]

`t78`	user	*Is anything on the heat already done and sitting?*
`t85`	assistant	**No—nothing is done and sitting. The fish in the top-wall pan has only just started cooking; its progress bar is still filling.**

[Intervening turns omitted.]

`t113`	user	*Give me a shout the moment the potato on the heat is done, would you?*
`t120`	assistant	**Yes—I'll watch for it. The potato has just started cooking; it isn't done yet.**
`t134`	assistant	**The potato is done now too. Scoop the fish first, then the potato from the central-island fryer.**

[Intervening turns omitted.]

`t169`	assistant	**You've picked up another potato, but both cooked potatoes are already on the finished plate. Trash the extra raw potato, then pick up the plate from the right-wall counter and take it to the serving pass to serve.**
`t169`	user	*The plate only has one potato on it.*
`t178`	assistant	**Sorry—I may have miscounted the portions. If there's only one potato on the plate, you're right to make the second. Finish chopping it, load the fryer and turn it on with empty hands, then retrieve the plate and scoop it when cooked.**
`t192`	user	*I want to modify the original recipe: add some fresh lettuce on top before serving.*
`t193`	event	The announced plan change is registered.
`t203`	assistant	**Lettuce would make the pass reject the plate—the order allows no extras. The second potato is done now: scoop it from the central-island fryer, then serve without lettuce.**
`t203`	user	*I'm adding the lettuce anyway.*

VHSim: the wrong object and a pause. A user with balanced preferences must take milk from the fridge, place it in the microwave, and switch the microwave on. GPT-Realtime 2.1 detects a mistaken pickup of juice and guides a correction. A later warning about stalled progress arrives after the user has resumed acting. The task completes at tick 88.

`t2`	event	Wrong-object event triggered near the fridge.
`t17-20`	action	`grab the juice`; the milk remains in the fridge.
`t24`	assistant	**I think you grabbed the wrong item. The milk is the white bottle that's still in the fridge. Please put the other bottle back if you're holding it, then pick up the milk.**
`t24`	user	*Oh, you're right, my bad. I'll swap them.*

t33-36	action	`put the juice in the fridge`; completed.
t36-48	event	A scheduled pause executes as `stand still` at the open fridge.
t48-49	action	`walk to the milk`; completed. The milk pickup begins at tick 49.
t50	assistant	**You seem paused there. I still see the milk in the fridge and nothing in your hand. Try picking up the milk (the white bottle) now; if something is blocking it, tell me what the house reports so we can adjust. Then close the fridge and head to the microwave.**
t50	user	*I've got it already, heading to the microwave now.*
t52	action	`grab the milk`; completed. The user's tick-50 claim preceded this completion.

[Intervening turns omitted.]

t58	action	`walk to the microwave`; rejected because the microwave is not visible.
t60	user	*Can't see it from here, let me turn around.*
t61-63	action	`turn left` $\times 3$; completed.
t74-77	action	`put the milk in the microwave`; completed.
t82-88	action	`switch on the microwave`; task completed.

ScreenSim: two incorrect settings changes and their repair. A novice user asks GPT-Realtime 2.1 to disable selected lock-screen access while preserving Control Center. One scheduled mistake disables Control Center; another changes *Type to Siri* instead of *Allow Siri When Locked*. The assistant identifies both errors and the user reverses them, reaching the requested configuration at tick 15.

t0	user	*I don't see Face ID on here.*
t0	user	*Should I scroll down?*
t1-2	action	`swipe up` twice; Face ID & Passcode becomes visible.

[Intervening turns omitted.]

t5	action	`tap Notification Center`; switches off the requested control.
t6	action	`tap Control Center`; the scheduled error disables a setting that should remain on.
t6	assistant	**Oops—looks like Control Center got toggled off. That's the one you want to keep on. Please turn Control Center back on, then switch off "Reply with Message." After that, we'll head to Siri settings to disable "Allow Siri When Locked."**
t6	user	*Oh, got it.*
t7	action	`tap Control Center`; restores the setting.

[Intervening turns omitted.]

t13	action	`tap Type to Siri`; the scheduled wrong-target error changes an unrelated setting.
t13	assistant	**That switched "Type to Siri" instead. Turn "Type to Siri" back off, then toggle "Allow Siri When Locked" off. Once that's off, you're all set.**
t13	user	*Oh, okay.*
t14	action	`tap Type to Siri`; reverses the extra change.
t15	action	`tap Allow Siri When Locked`; task completed.